
Plataforma de Badges da Softinsa

Relatório da Unidade Curricular de Projeto Integrado

Curso de Licenciatura em Engenharia Informática

Ano Letivo: 2.º

Hugo Alexandre Pereira Afonso (pv30032)

Mateus Valente Frias Silva (pv29989)

Rodrigo de Almeida Martins (pv30773)

Susana Daniela Tavares (pv27467)

Trabalho efetuado sob a orientação de:

Artur Sousa

Júlio Florentino

Junho 2026

Agradecimentos

Enquanto grupo, gostaríamos de manifestar o nosso sincero agradecimento aos professores Artur Sousa e Júlio Florentino pelo apoio, orientação e tempo que nos dedicaram ao longo deste trabalho.

Gostaríamos também de expressar a nossa gratidão à Softinsa, bem como a todos os que a representaram, principalmente o engenheiro Rui Costa, pelo tempo investido na elaboração do enunciado deste projeto.

Resumo

Palavras-chave:

Abstract

Keywords:

Lista de Abreviaturas

API	Application Programming Interface
AWS	Amazon Web Services
BD	Base de Dados
IBM	International Business Machines
JWT	JSON Web Token
KPI	Key Performance Indicator
LP	Learning Path
RGPD	Regulamento Geral sobre a Proteção de Dados
SL	Service Line
SLA	Service Level Agreement
SLL	Service Line Leader
TM	Talent Manager
UC	Unidade Curricular
UI	User Interface
URL	Uniform Resource Locator
UX	User Experience

Glossário

Badge	Credencial digital verificável que atesta a obtenção de um determinado nível de competências ou a conclusão de requisitos específicos por parte do consultor.
Consultor	Perfil de utilizador da plataforma que representa o colaborador/técnico da Softinsa responsável por submeter evidências e progredir nos itinerários de aprendizagem.
Credly	Plataforma global de referência no mercado para a emissão, gestão e partilha de badges e certificações digitais.
Evidência	Documento digital (certificado, diploma, relatório em formato PDF, imagem ou outro) submetido pelo consultor que comprova a realização de um determinado requisito.
Gamificação	Aplicação de dinâmicas, mecânicas e elementos típicos de jogos (como sistema de pontos, conquistas especiais e rankings) num contexto corporativo para motivar a aprendizagem.
Learning Path	Estrutura macro de formação contínua dentro da empresa, que agrupa várias SLs e áreas técnicas.
Requisito	Meta ou curso específico associado a um determinado nível, cuja conclusão e validação são obrigatórias para obter o respetivo badge.
Service Line	Divisão interna da empresa que agrupa áreas de competências técnica afins.
Softinsa	Empresa do grupo IBM em Portugal, focada em serviços de consultoria tecnológica e gestão de aplicações, que forneceu o tema para este projeto integrado.
Workflow de Estados	Ciclo de vida sequencial pelo qual uma candidatura a um badge transita no sistema (compreendendo os estados ‘Open’, ‘Submitted’, ‘Em Validação’ e ‘Fechado’) consoante as ações dos avaliadores.

Índice

Lista de Figuras

Lista de Tabelas

1

Introdução

O presente relatório documenta todo o processo de idealização, planejamento e implementação da Plataforma de Badges da Softinsa, a propósito da Unidade Curricular (UC) de Projeto Integrado. O tema do projeto foi concebido e fornecido pela Softinsa, com o propósito de desafiar os estudantes a desenvolverem uma solução tecnológica robusta e adaptada a uma necessidade real do contexto corporativo da empresa. Sendo o desenvolvimento deste ecossistema efetuado integralmente a partir do zero, este documento detalha as várias etapas do ciclo de vida do software, compreendendo a modelação conceptual do modelo de dados, a conceção da experiência e interface do utilizador (UI/UX) através de protótipos em Figma [figma], e a subsequente codificação das três frentes da solução: a API REST [restapi] de backend, o portal Web e a aplicação Mobile (apenas para o perfil de consultor).

1.1 Enquadramento e Motivação

No atual panorama do setor das tecnologias da informação, a competitividade das empresas está intrinsecamente ligada à capacidade de valorização e atualização contínua do seu capital humano. Como empresa líder em consultoria tecnológica, a Softinsa reconhece que a formação contínua e a certificação de competências dos seus colaboradores são pilares fundamentais para sustentar o crescimento do negócio, responder às exigências do mercado e manter equipas de alto rendimento. Para organizar este esforço de capacitação, a empresa dispõe de múltiplos Learning Paths (LP), ou itinerários de aprendizagem, estruturados por Service Lines (SL) e áreas técnicas específicas, oferecendo uma progressão de carreira clara através de vários níveis de consolidação de conhecimento.

Contudo, a gestão manual e descentralizada destas formações – frequentemente realizadas em plataformas externas como a Udemy, IBM, AWS ou Microsoft – acarreta desafios operacionais significativos. Identificou-se uma clara dificuldade em evidenciar e validar de forma ágil as competências efetivamente adquiridas pelos consultores. Para responder a isto, o ecossistema da Softinsa exige um fluxo rigoroso de auditoria assente em papéis bem definidos:

o Consultor, que executa as formações e submete as suas evidências; o Talent Manager (TM), responsável por uma primeira validação global e transversal dos documentos submetidos; e o Service Line Leader (SLL), que efetua a validação técnica e final com base na sua área específica de especialização. Todo este ecossistema é centralizado e parametrizado pelo perfil de Administrador, que faz a gestão de conteúdos e utilizadores da plataforma.

Atualmente, a ausência de um sistema automatizado e padronizado que una estes perfis não só reduz a visibilidade interna do leque de competências disponíveis, como também limita o impacto reputacional que certificações verificáveis poderiam trazer junto de clientes e parceiros. Aliado a isto, a falta de uma dinâmica de gamificação no ecossistema corporativo traduz-se numa oportunidade perdida para motivar os colaboradores a assumirem um papel mais proativo na sua própria evolução profissional. Os consultores necessitam de estímulos visuais claros, reconhecimento de marcos alcançados e uma perceção tangível do seu progresso dentro da organização.

Foi neste contexto que surgiu a necessidade de desenvolver a presente plataforma de badges digitais. Inspirada em soluções de referência como o Credly [**credly**] e no standard Open Badges [**openbadges**], esta ferramenta visa colmatar as falhas identificadas através da centralização do fluxo de submissão e auditoria de evidências entre os diferentes intervenientes, da introdução de mecânicas de recompensa por pontos [**gamification**] e do fortalecimento de uma cultura colaborativa e transparente dentro da Softinsa.

1.2 Objetivos

O objetivo geral deste projeto consiste no desenvolvimento, a partir do zero, de um ecossistema tecnológico integrado para a gestão de badges digitais da Softinsa, que replique e adapte as boas práticas de plataformas de referência de mercado [**credly**, **openbadges**]. A solução visa aproximar a empresa da sua rede de colaboradores de forma colaborativa e ágil, racionalizando os processos de validação de competências.

Para concretizar este propósito, foram definidos os seguintes objetivos específicos (para além dos fornecidos pela própria Softinsa no enunciado), segmentados por vertentes técnicas e operacionais:

1.2.1 Desenvolvimento Tecnológico e Arquitetural

- **Conceção do Modelo de Dados:** Desenhar e implementar uma base de dados estruturada e escalável, capaz de suportar o fluxo de estados das candidaturas e que permita, futuramente, a expansão para novos LPs (como o de Power Skills);
- **Prototipagem:** Desenvolvimento de um Protótipo de Alta Fidelidade (PAF) em Figma [**figma**], de forma a explorar a melhor solução visual (UI) e também no que toca à experiência do utilizador (UX);

- **Implementação da API REST (Backend):** Desenvolver o motor lógico central do sistema em Node.js [**nodejs**] com Express [**expressjs**] e base de dados PostgreSQL [**postgresql**], para garantir a segurança dos dados, a autenticação robusta de utilizadores e a comunicação fluida com as interfaces;
- **Construção do Portal Web:** Desenhar e codificar um portal responsivo em React [**react**] com Bootstrap [**bootstrap**], dedicado aos perfis de gestão (Administrador, TM, SLL) e também de consulta para o consultor, com foco na apresentação de métricas, relatórios de auditoria e ferramentas de parametrização;
- **Desenvolvimento da Aplicação Mobile:** Desenvolver uma aplicação móvel em Flutter [**flutter**], focada na experiência do consultor, permitindo-lhe gerir o seu perfil, acompanhar o seu progresso visual e submeter evidências de formação de forma intuitiva.

1.2.2 Funcionalidades Operacionais e de Negócio

- **Automatização do Workflow de Aprovação:** Digitalizar o ciclo de vida de uma candidatura, garantindo o registo auditável e o histórico de decisões e comentários entre o consultor, TM e SLL;
- **Introdução de Mecânicas de Gamificação:** Implementar um sistema motivacional assente na atribuição de pontos por badges obtidos e na celebração de conquistas especiais para incentivar a formação contínua;
- **Garantia de Transparência e Verificabilidade:** Criar um sistema de verificação pública através de ligações para cada badge, permitindo aos consultores partilharem as suas competências certificadas de forma credível em plataformas como o LinkedIn.

1.3 Estrutura do Relatório

2

Requisitos do Sistema e Abordagem

Neste capítulo é apresentada a análise dos requisitos do sistema, resultante da interpretação do enunciado fornecido pela Softinsa e da sua decomposição sob a ótica da engenharia de software. Os requisitos foram categorizados em funcionais e não funcionais, sendo os primeiros organizados por perfil de utilizador e plataforma, e os segundos agrupados por preocupações transversais ao sistema.

2.1 Requisitos Funcionais (RF)

Os requisitos funcionais descrevem as ações concretas que o sistema permite realizar, tendo sido identificados a partir do enunciado do projeto e refinados ao longo do processo de desenvolvimento. Apresentam-se segmentados pelos quatro perfis de utilizador da plataforma Web e pelo perfil exclusivo da aplicação Mobile.

2.1.1 Consultor (Plataforma Web)

O consultor constitui o ator central do ecossistema. As funcionalidades que lhe são destinadas na plataforma Web abrangem todo o ciclo de interação com os badges, desde o registo inicial até à partilha pública das credenciais obtidas.

- Receção de e-mail de confirmação de registo e alteração obrigatória da password no primeiro acesso;
- Seleção da área de interesse no momento do registo, para apresentação preferencial dos badges correspondentes;
- Consulta de todos os badges disponíveis na plataforma, incluindo os de outras áreas;
- Dashboard pessoal com visualização do progresso nos Learning Paths;

- Sistema de upload de evidências (certificados, diplomas, relatórios) para comprovar o cumprimento dos requisitos;
- Visualização em tempo real do estado dos pedidos de badges submetidos;
- Consulta do histórico completo de badges obtidos e em processo;
- Catálogo de badges disponíveis acompanhado de descrições detalhadas e respectivos requisitos;
- Aceitação dos termos de RGPD para publicação e partilha de badges;
- Partilha de badges obtidos diretamente no LinkedIn;
- Sistema de pontos acumulados por badges obtidos e visualização de conquistas especiais;
- Métricas de progresso visual e celebração de marcos alcançados;
- Recomendações de próximos badges com base no histórico do consultor;
- Download de certificados personalizados em formato PDF;
- Receção de notificações de aprovação, rejeição, confirmação de candidatura e alertas de expiração de badges;
- Galeria pública de badges obtidos com página individual acessível por URL única;
- Sistema de verificação pública por ligação única para cada badge, permitindo a comprovação externa da certificação.

2.1.2 Service Line Leader

O Service Line Leader (SLL) é responsável pela validação final das candidaturas dentro da sua Service Line. O seu âmbito de atuação é restrito aos consultores e pedidos associados à sua área de especialização.

- Consulta de todos os badges disponíveis na plataforma, incluindo os de áreas externas à sua;
- Dashboard com o progresso agregado de todos os consultores da sua Service Line;
- Visualização em tempo real do estado dos pedidos da sua área e consulta do respetivo histórico;
- Acesso ao catálogo de badges com descrições e requisitos detalhados;

- Visualização do sistema de pontos por badges atribuídos na sua área;
- Acesso a badges de conquistas especiais (*Badges Premium*);
- Geração de relatórios de badges atribuídos por área e período;
- Exportação de dados (pedidos, badges, consultores e aprovações) para Excel e PDF;
- Download e geração de certificados de badges em formato PDF;
- Receção de notificações por e-mail relativas a candidaturas e validações pendentes;
- Visualização e comparação do ranking de consultores da sua Service Line;
- Consulta do histórico detalhado associado a cada processo de candidatura.

2.1.3 Talent Manager

O Talent Manager (TM) desempenha o papel de primeiro validador global das evidências submetidas, atuando de forma transversal a todas as Service Lines e áreas.

- Consulta de todos os badges disponíveis na plataforma, com acesso ao catálogo, descrições e requisitos;
- Dashboard com o progresso de todos os consultores sob a sua gestão;
- Sistema de verificação e validação das evidências submetidas pelos consultores;
- Visualização em tempo real do estado de todos os pedidos e consulta do respetivo histórico;
- Geração de relatórios e exportação de dados (pedidos, badges, consultores, aprovações e rejeições) para Excel e PDF;
- Visualização do sistema de pontos e de conquistas especiais;
- Download de certificados personalizados em formato PDF;
- Receção de notificações por e-mail relativas a pedidos de candidatura;
- Visualização de badges próximos da data de expiração;
- Consulta do histórico completo associado a cada processo de candidatura.

2.1.4 Administrador

O Administrador é o gestor máximo da plataforma, com controlo total sobre utilizadores, conteúdos, parametrização e monitorização global do sistema.

- Gestão completa de utilizadores: criação de contas, atribuição de perfis (Consultor, TM, SLL) e gestão de permissões;
- Gestão da estrutura hierárquica de conteúdos: criação, edição e remoção de Learning Paths, Service Lines, Áreas, Níveis e Requisitos;
- Gestão de badges: definição de pontos, regras de expiração e configuração de requisitos por nível;
- Exportação global de dados para Excel e PDF;
- Configuração das notificações do sistema;
- Gestão de políticas de privacidade e conformidade com o RGPD;
- Consulta e gestão global de todos os pedidos de badges (atribuídos e em curso);
- Criação e gestão de informações genéricas e avisos (ativos e inativos) para comunicação com todos os utilizadores;
- Definição e gestão de SLAs de resposta para as equipas de validação (bónus);
- Envio de notificações por e-mail e PUSH em caso de incumprimento de SLAs (bónus).

2.1.5 Consultor (Aplicação Mobile)

A aplicação móvel foi desenvolvida em Flutter [**flutter**] e Dart [**dart**], exclusivamente para o perfil de Consultor, replicando as funcionalidades essenciais da plataforma Web numa experiência adaptada ao contexto móvel, com arquitetura *offline-first* suportada por uma base de dados local SQLite [**sqflite**] e notificações via Firebase Cloud Messaging [**firebase**].

- Registo e autenticação com alteração obrigatória de password no primeiro acesso;
- Escolha da área de interesse para personalização da experiência;
- Consulta do catálogo completo de badges, incluindo os de outras áreas;
- Dashboard pessoal com visualização do progresso nos Learning Paths;
- Sistema de upload de evidências para submissão de candidaturas;
- Visualização em tempo real do estado dos pedidos e consulta do histórico;

- Aceitação dos termos de RGPD e partilha de badges no LinkedIn;
- Visualização de pontos acumulados, conquistas especiais e métricas de progresso;
- Celebração de marcos alcançados e recomendações de próximos badges;
- Download de certificados personalizados em formato PDF;
- Receção de notificações de aprovação, rejeição e alertas de expiração;
- Acesso à página pública de cada badge e verificação por ligação única.

2.1.6 Indicadores e Relatórios (Mínimo Exigível)

O enunciado define um conjunto mínimo de indicadores-chave de desempenho (KPIs) que a plataforma deve disponibilizar nos dashboards de gestão. A Tabela 2.1 sintetiza estes indicadores.

Indicador	Descrição
% Mensal	Percentagem de badges atribuídos com visão mensal.
N.º por datas	Número de badges atribuídos por intervalo de datas.
N.º por Learning Path	Número de badges por Learning Path.
N.º por nível	Número de badges por nível dos Learning Paths.
N.º de utilizadores	Número total de utilizadores registados na plataforma.

Tabela 2.1: Indicadores mínimos exigidos nos dashboards.

2.1.7 Diagramas de Casos de Uso

Para complementar a especificação textual dos requisitos funcionais, foram elaborados diagramas de casos de uso UML para cada perfil de utilizador. Os diagramas encontram-se segmentados por grupos funcionais de modo a garantir a legibilidade, mantendo a rastreabilidade com as funcionalidades descritas nas secções anteriores.

Consultor

O consultor, enquanto ator principal do sistema, interage com seis grupos funcionais distintos. O primeiro diagrama (Figura 2.1) cobre as operações de autenticação, gestão de perfil e exploração do catálogo de badges. O segundo (Figura 2.2) abrange o fluxo de candidaturas e o módulo de gamificação e progresso. O terceiro (Figura 2.3) apresenta as funcionalidades de partilha de credenciais e notificações.

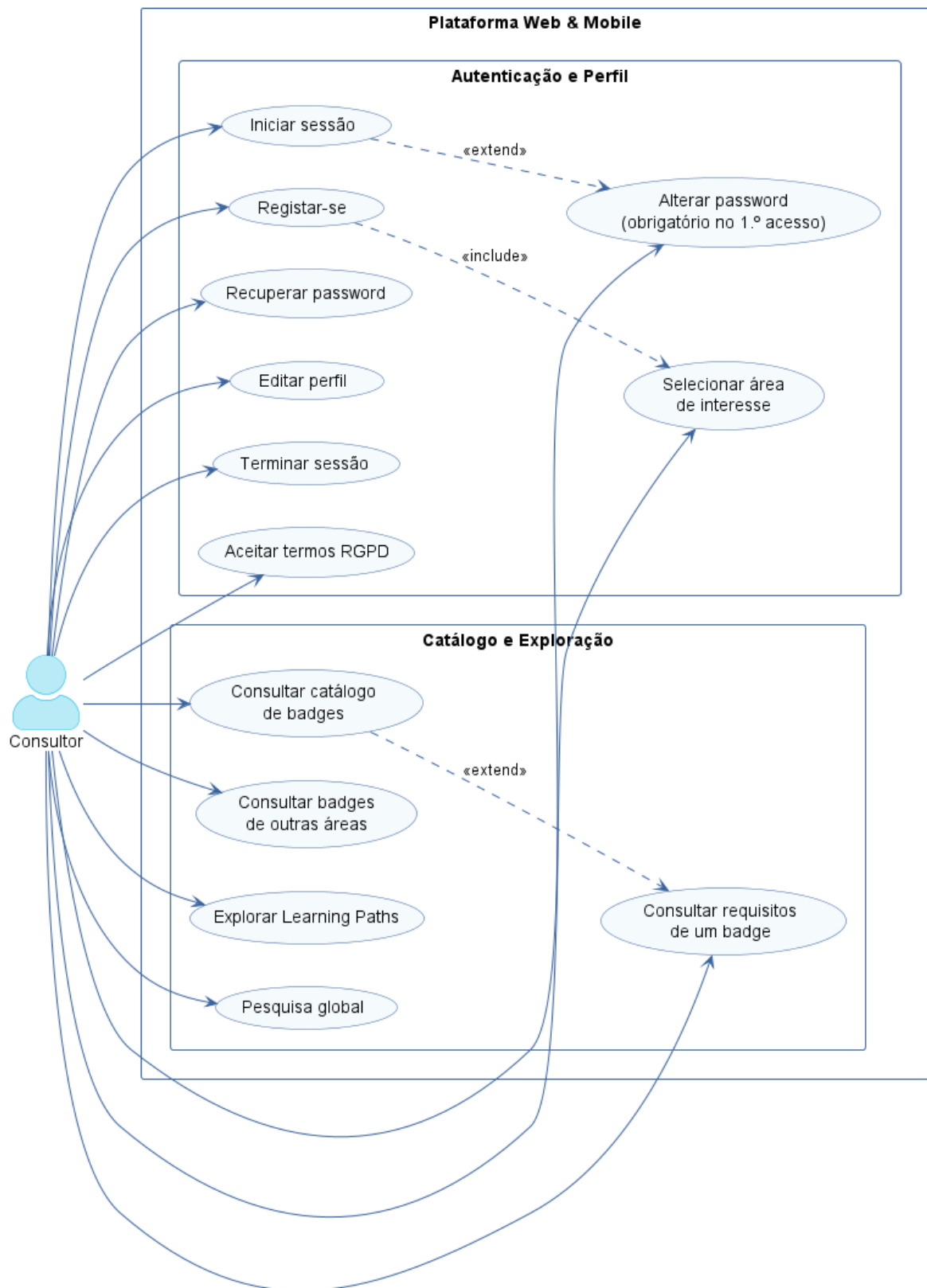
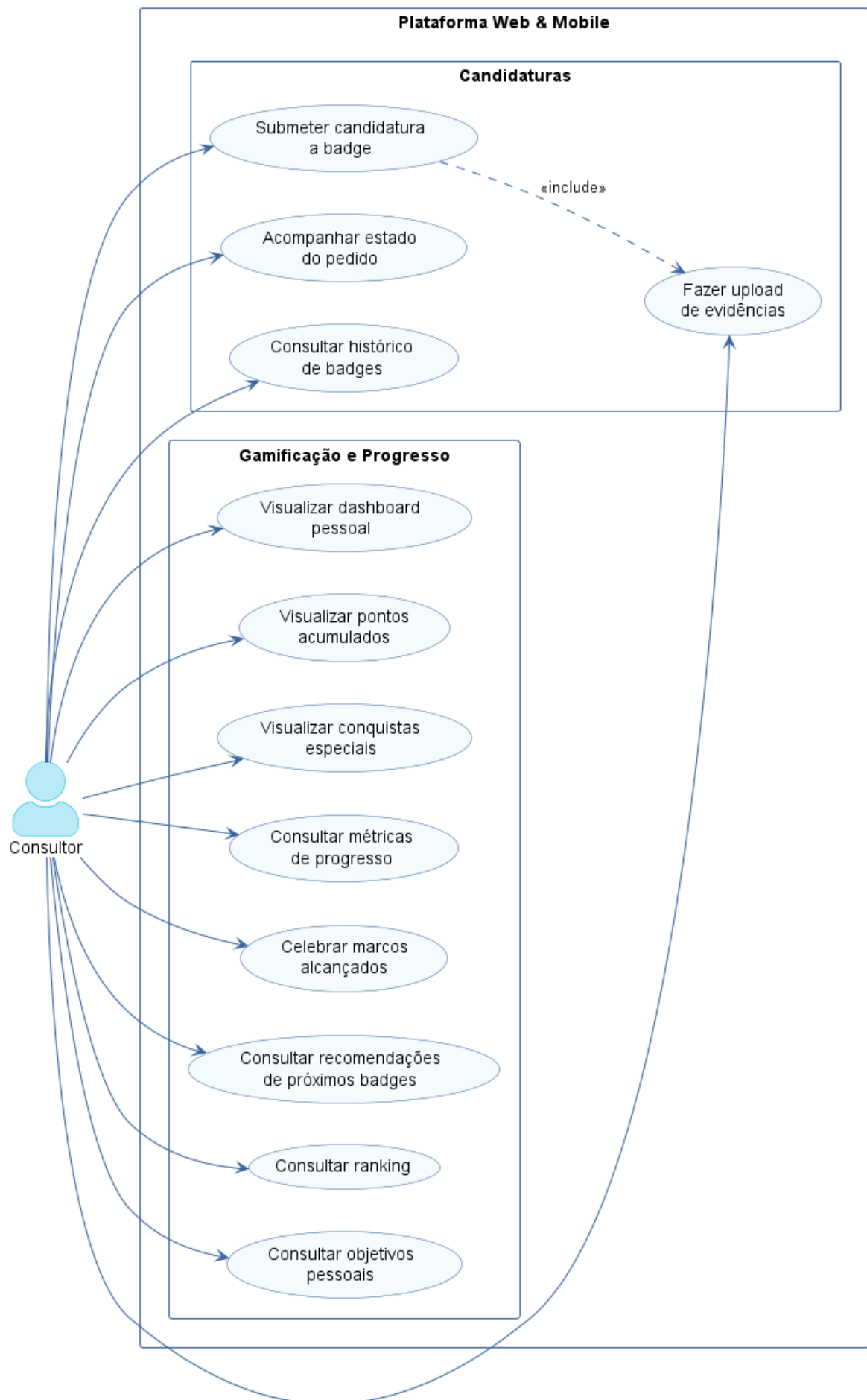


Figura 2.1: Casos de uso do Consultor — Autenticação e Catálogo.



Plataforma de Badges da Softinsa

Figura 2.2: Casos de uso do Consultor — Candidaturas e Gamificação.

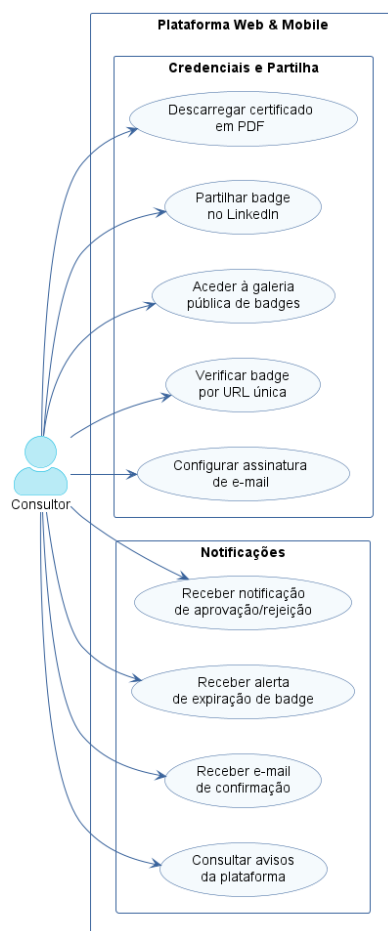


Figura 2.3: Casos de uso do Consutor — Credenciais e Notificações.

Service Line Leader

O SLL atua como validador final das candidaturas, com atuação restrita à sua Service Line. A Figura 2.4 apresenta as operações de autenticação e consulta do catálogo. A Figura 2.5 detalha o fluxo de validação de candidaturas e o acesso ao dashboard da sua área. A Figura 2.6 cobre as funcionalidades de exportação de relatórios e notificações.

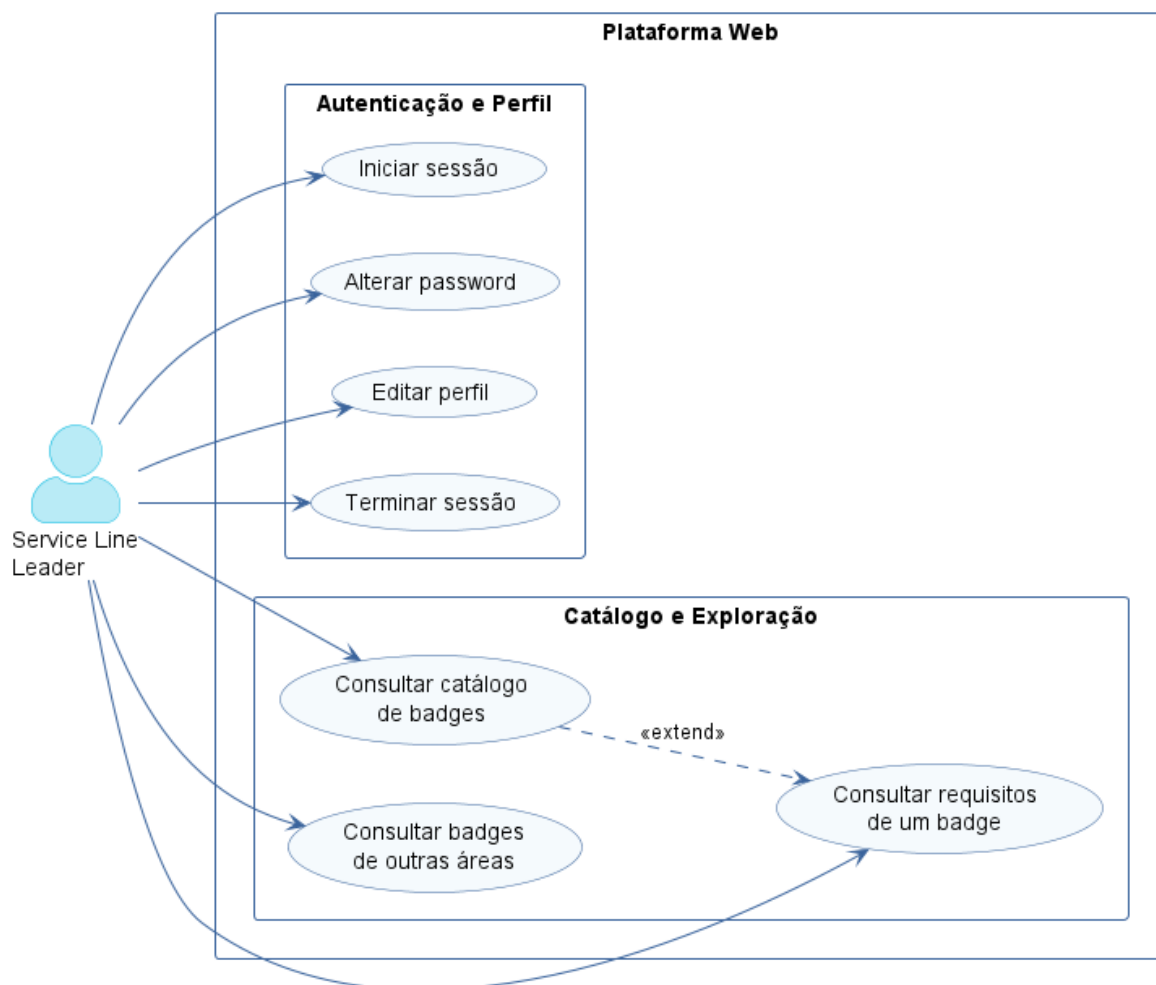


Figura 2.4: Casos de uso do Service Line Leader — Autenticação e Catálogo.

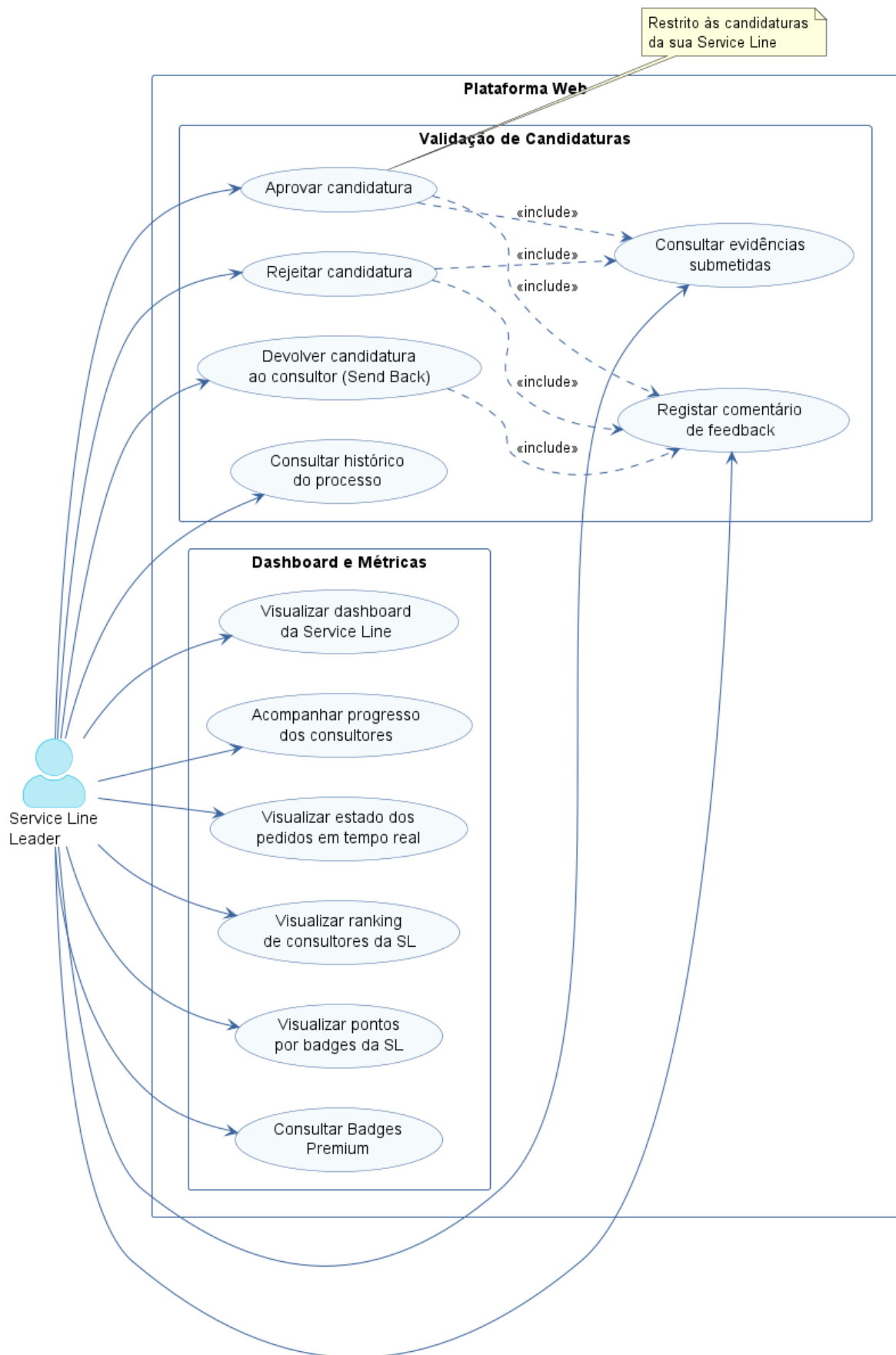


Figura 2.5: Casos de uso do Service Line Leader — Validação e Dashboard.

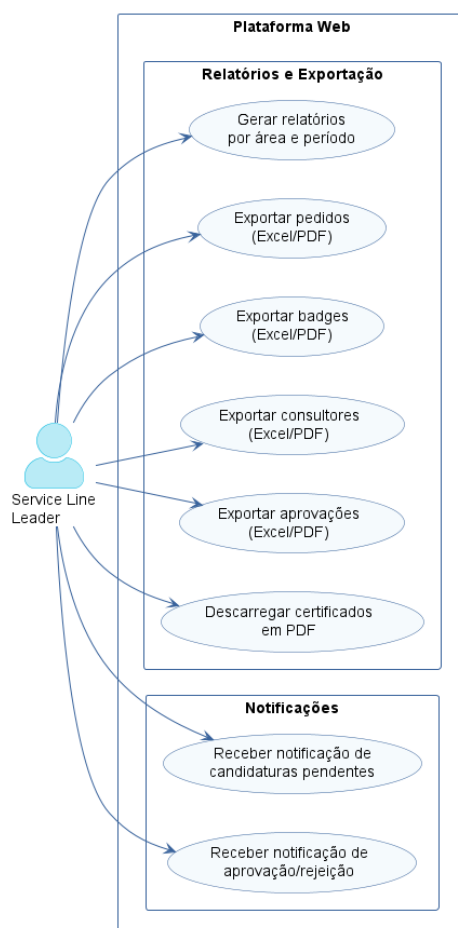


Figura 2.6: Casos de uso do Service Line Leader — Relatórios e Notificações.

Talent Manager

O TM desempenha o papel de primeiro validador, com atuação transversal a todas as Service Lines. A Figura 2.7 ilustra as operações de autenticação e exploração do catálogo. A Figura 2.8 detalha o fluxo de triagem e validação de evidências, bem como o dashboard global. A Figura 2.9 apresenta as funcionalidades de exportação e notificações.

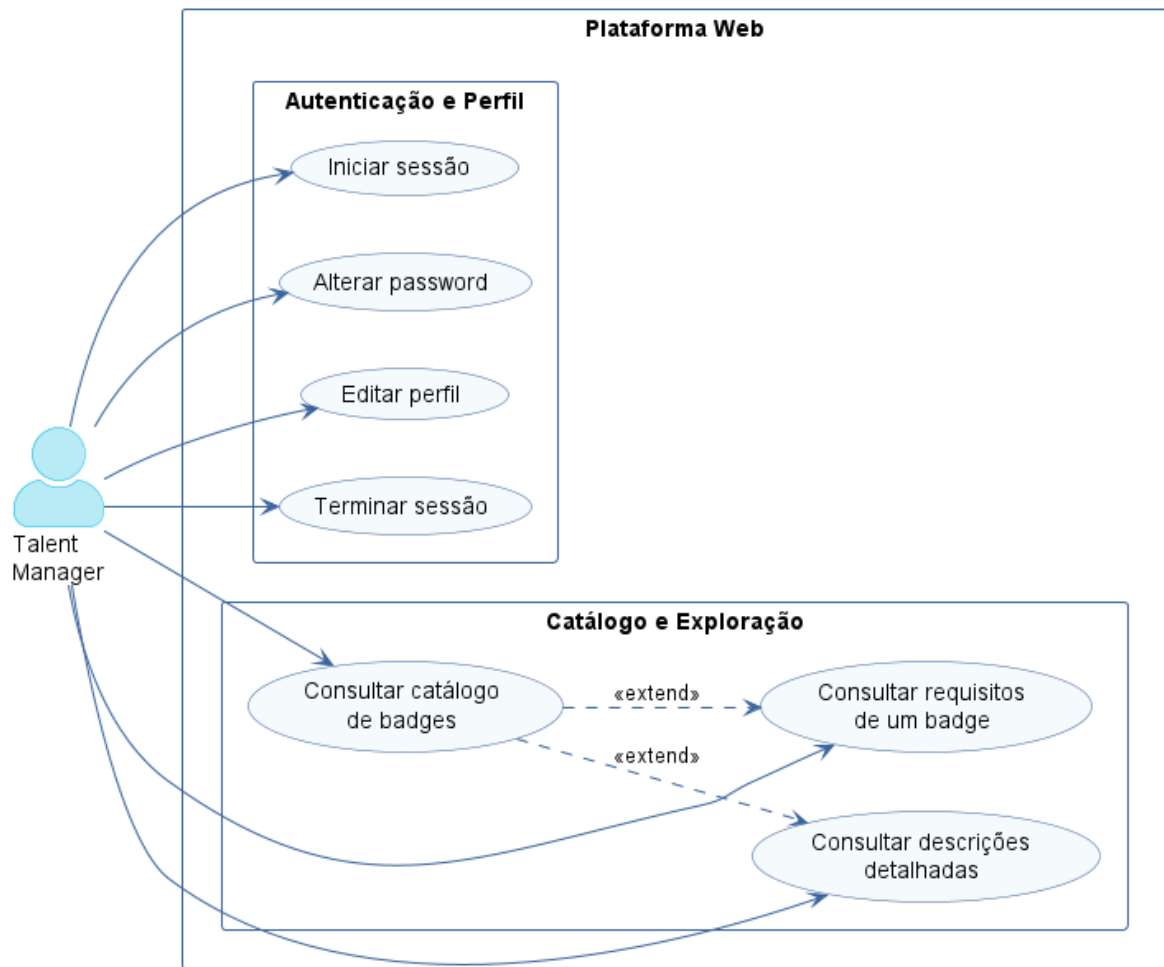


Figura 2.7: Casos de uso do Talent Manager — Autenticação e Catálogo.

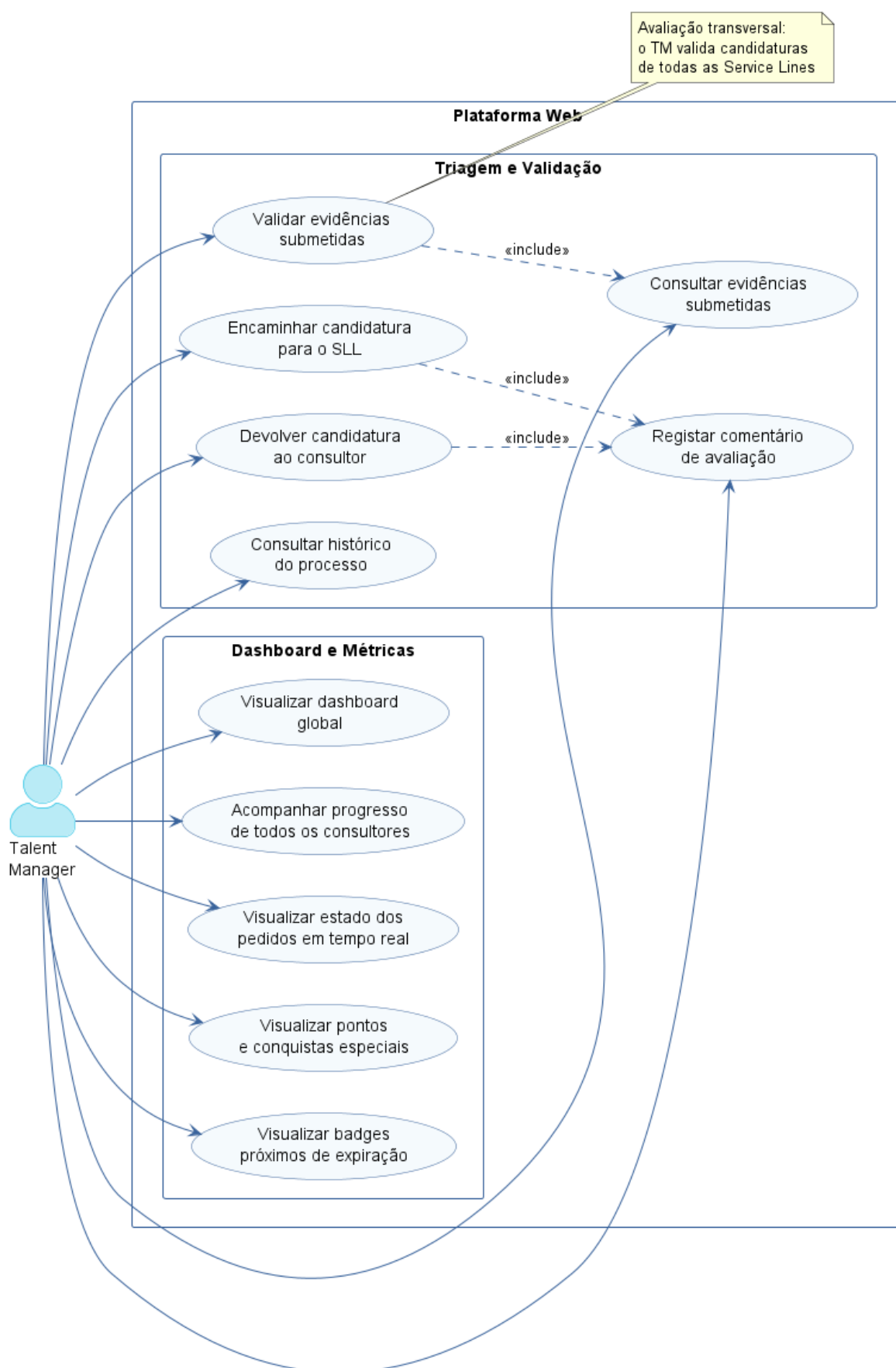


Figura 2.8: Casos de uso do Talent Manager — Triagem e Dashboard.

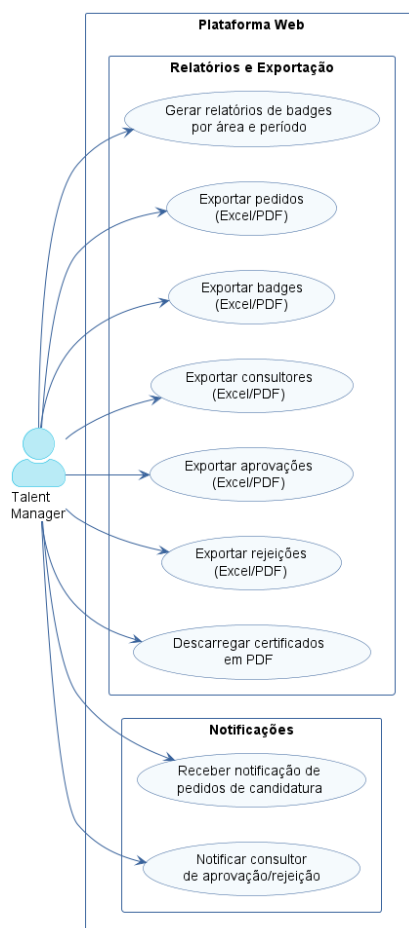
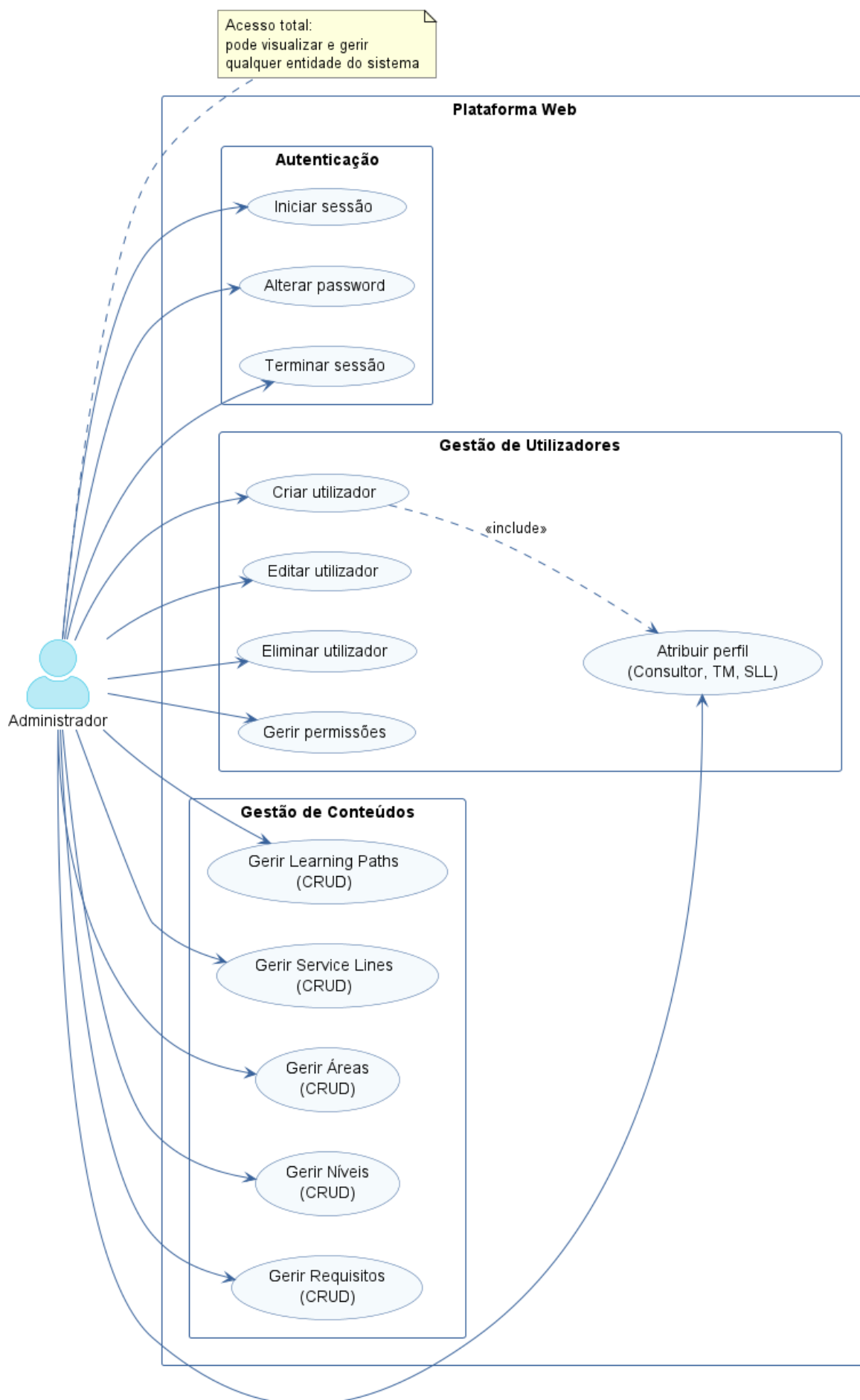


Figura 2.9: Casos de uso do Talent Manager — Relatórios e Notificações.

Administrador

O Administrador dispõe de acesso total à plataforma, sendo responsável pela gestão de utilizadores, conteúdos e parametrização global. A Figura 2.10 abrange a autenticação, a gestão de utilizadores e a gestão da hierarquia de conteúdos. A Figura 2.11 detalha a gestão de badges e o módulo de comunicação e avisos. A Figura 2.12 apresenta as funcionalidades de conformidade, SLAs e exportação de dados.



Plataforma de Badges da Softinsa

Figura 2.10: Casos de uso do Administrador — Utilizadores e Conteúdos.

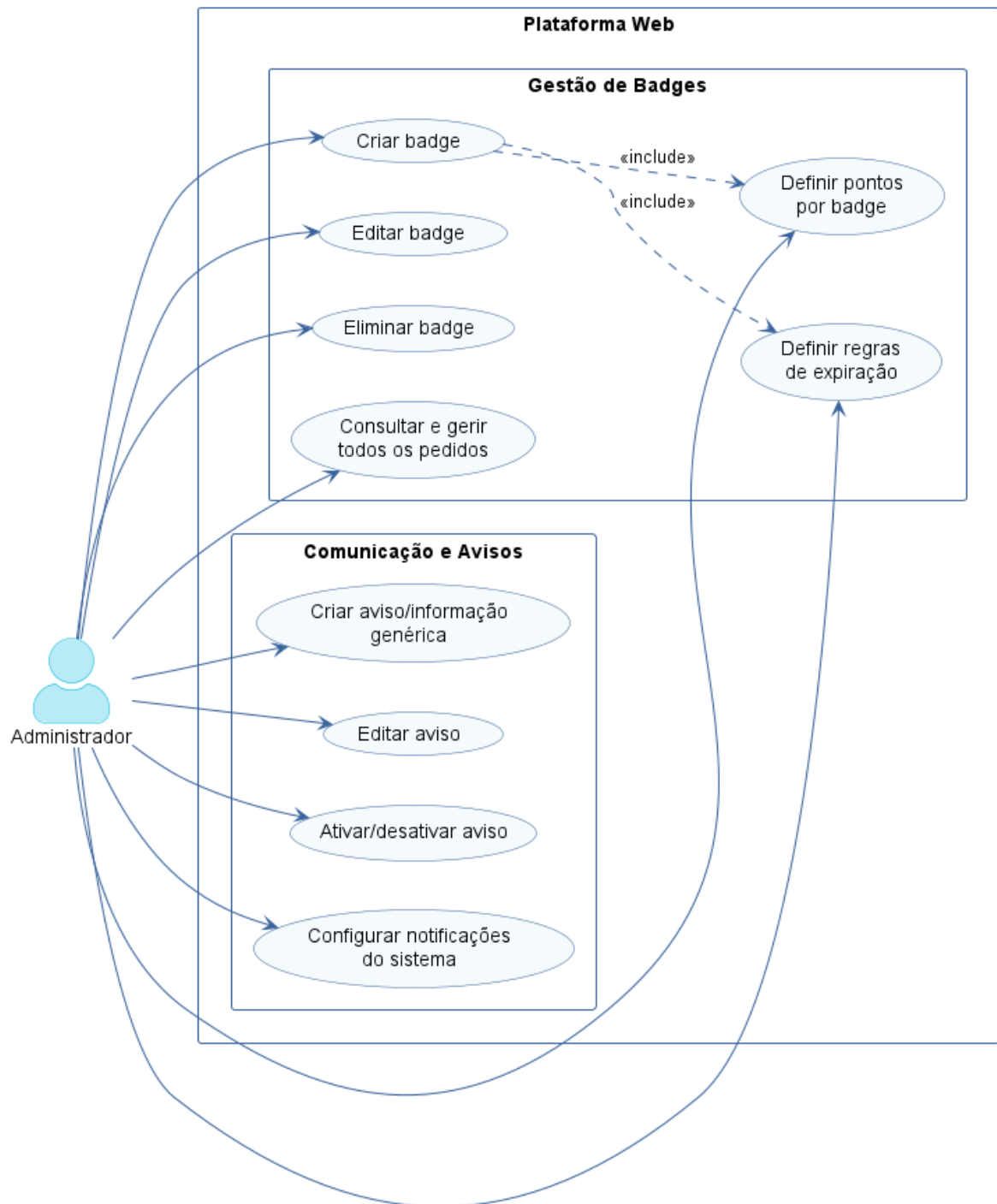


Figura 2.11: Casos de uso do Administrador — Badges e Comunicação.

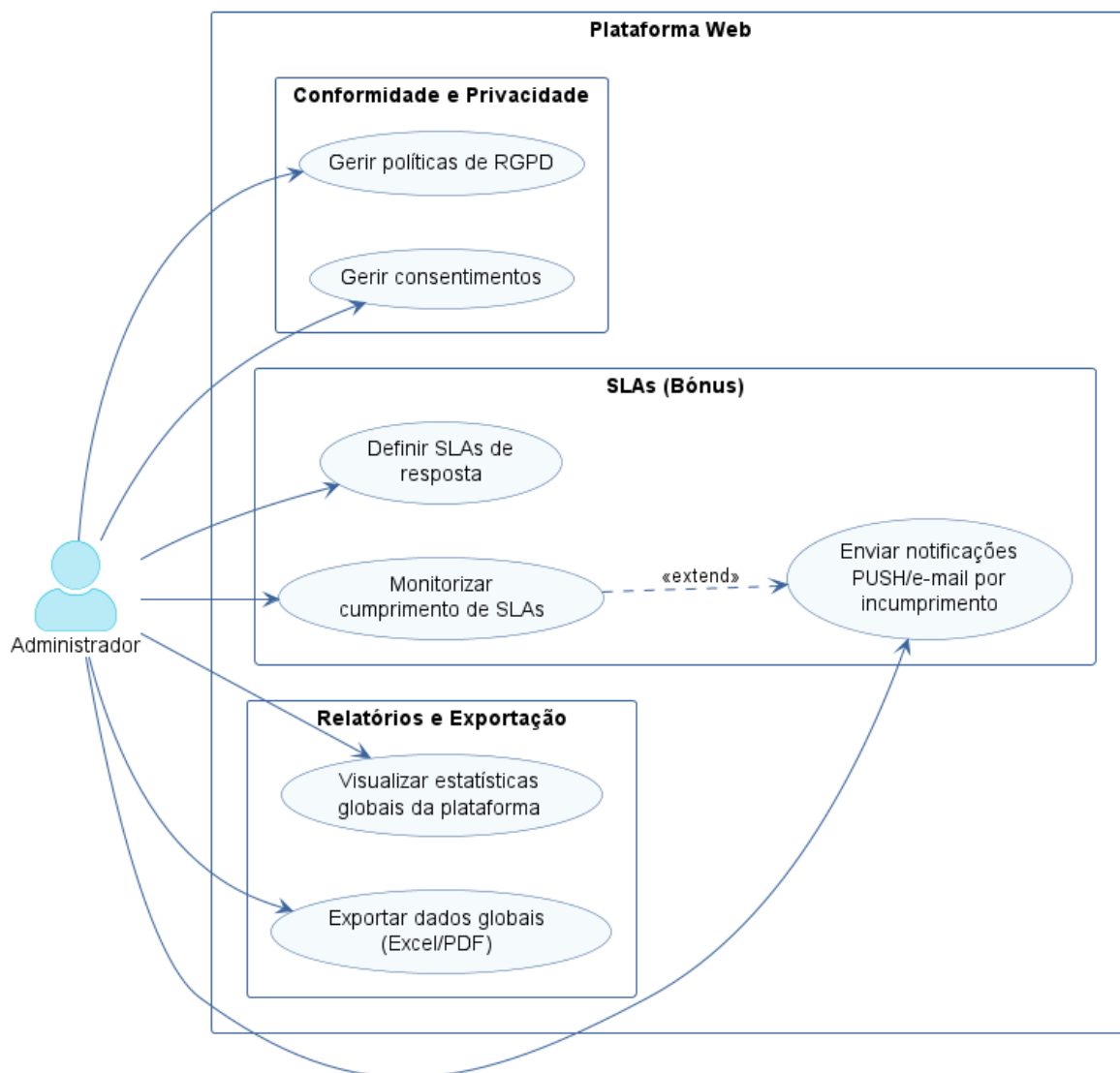


Figura 2.12: Casos de uso do Administrador — Conformidade, SLAs e Relatórios.

2.2 Requisitos Não Funcionais (RNF)

Os requisitos não funcionais definem as propriedades transversais de qualidade que o sistema deve garantir, independentemente das funcionalidades específicas de cada perfil. Foram identificadas quatro grandes áreas de preocupação: segurança, usabilidade, escalabilidade e internacionalização.

2.2.1 Segurança

A segurança foi tratada como uma prioridade estrutural desde o desenho da arquitetura até à implementação dos diferentes módulos. As medidas adotadas abrangem a comunicação, a autenticação, a autorização e a proteção dos dados pessoais.

- **Encriptação de transporte:** Toda a comunicação entre o cliente e o servidor é realizada obrigatoriamente por HTTPS [**restapi**], assegurada por terminação TLS ao nível do *reverse proxy* de produção;
- **Autenticação por tokens:** O sistema utiliza JSON Web Tokens (JWT) [**jwt**] com o algoritmo HS256 para autenticação. O fluxo inclui *access tokens* de curta duração e *refresh tokens* para renovação transparente de sessões;
- **Alteração obrigatória de password:** No primeiro acesso após a criação da conta, o sistema força a alteração da password — armazenada com hashing bcrypt [**bcrypt**] — através de uma *flag* dedicada (*fpc* — *force password change*) que bloqueia qualquer outra operação até à sua resolução;
- **Cabeçalhos de segurança:** A API utiliza o *middleware* Helmet [**helmet**] para definir automaticamente cabeçalhos HTTP de proteção contra ataques comuns (XSS, *clickjacking*, *MIME sniffing*) [**owasp**];
- **Controlo de origens (CORS):** O acesso à API é restrito a uma lista explícita de origens autorizadas, impedindo pedidos de domínios não reconhecidos;
- **Limitação do corpo dos pedidos:** O tamanho máximo dos corpos JSON aceites pela API está limitado a 100 KB, prevenindo ataques de negação de serviço por *payloads* volumosos. Os ficheiros de evidências são carregados diretamente para o serviço de armazenamento externo Supabase [**supabase**], sem transitar pelo corpo dos pedidos;
- **Validação de dados de entrada:** Todos os dados recebidos pela API são validados através de esquemas Zod [**zod**] modulares, garantindo a conformidade dos parâmetros antes de qualquer processamento. Esta validação é replicada no lado do cliente para *feedback* imediato ao utilizador;
- **Controlo de acesso baseado em papéis (RBAC):** Cada *endpoint* da API está protegido por *middleware* de verificação de papéis, assegurando que cada perfil (Administrador, Consultor, TM, SLL) apenas acede às operações que lhe são permitidas;
- **Conformidade com o RGPD:** O sistema dispõe de um módulo dedicado à gestão de consentimentos, exportação de dados pessoais e eliminação de contas, em conformidade com o Regulamento Geral sobre a Proteção de Dados [**rgpd**].

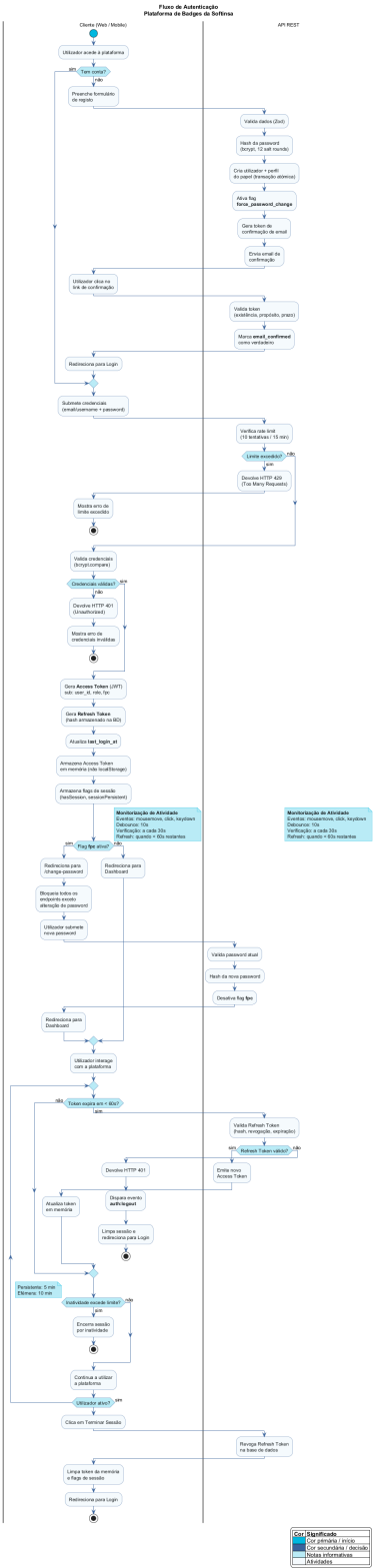


Figura 2.13: Fluxo completo de autenticação da plataforma.

2.2.2 Usabilidade e Responsividade

A interface do utilizador foi concebida para oferecer uma experiência consistente e intuitiva em qualquer dispositivo, desde ecrãs *desktop* de grande dimensão até dispositivos móveis compactos.

- **Design responsivo:** O portal Web utiliza o sistema de grelha do Bootstrap [**bootstrap**] complementado por *media queries* CSS específicas, com pontos de quebra definidos para garantir a adaptação progressiva do *layout* até uma largura mínima de 300 px;
- **Arquitetura de componentes:** Cada componente da interface é desenvolvido num diretório próprio, contendo o ficheiro JSX e o respetivo módulo CSS (`.module.css`), promovendo a reutilização e a manutenibilidade do código;
- **Sistema de design:** As cores, tipografia e espaçamentos são centralizados num ficheiro de variáveis CSS globais (`colors.css`), garantindo coerência visual em toda a aplicação e facilitando alterações futuras à identidade gráfica;
- **Dashboards diferenciados por perfil:** Cada papel de utilizador dispõe de um dashboard dedicado, adaptado às suas necessidades específicas e apresentando apenas a informação e as ações relevantes para o seu contexto;
- **Validação visual de campos:** Os campos de formulário com dados inválidos ou em falta são assinalados com uma identificação a vermelho, acompanhada de mensagens de erro específicas por campo, conforme definido no enunciado;
- **Orientação fixa na aplicação móvel:** A aplicação Mobile está bloqueada em modo retrato, assegurando uma experiência de utilização otimizada e consistente em todos os dispositivos móveis;
- **Acessibilidade:** As animações da interface respeitam a preferência do sistema operativo `prefers-reduced-motion`, reduzindo ou eliminando transições para utilizadores que assim o configurem.

2.2.3 Escalabilidade

A arquitetura do sistema foi desenhada com vista à expansão futura, tanto ao nível dos conteúdos como da base de utilizadores.

- **Modelo de dados hierárquico e extensível:** A base de dados organiza os conteúdos numa estrutura hierárquica de Learning Paths, Service Lines, Áreas, Níveis e Requisitos. Esta modelação permite a introdução de novos itinerários de aprendizagem (como o futuro LP de *Power Skills*) sem necessidade de alterações ao esquema relacional;

- **Encaminhamento hierárquico na API:** A API suporta rotas aninhadas que refletem diretamente a hierarquia do modelo de dados, permitindo o acesso contextualizado a qualquer nível da árvore de conteúdos;
- **Modularidade da base de código:** A API, construída sobre Express [`expressjs`] e o ORM Sequelize [`sequelize`], está organizada em módulos independentes (rotas, controladores, serviços e validações), facilitando a adição de novas funcionalidades sem impacto nas existentes;
- **Processos de fundo:** Tarefas recorrentes como a monitorização de SLAs, a verificação de expiração de badges e o envio de lembretes de objetivos são executadas por *workers* autónomos baseados em node-cron [`nodecron`], evitando a sobrecarga do ciclo de pedido-resposta principal.

2.2.4 Internacionalização

O suporte multilingue foi implementado de forma transversal a todas as camadas do sistema, abrangendo tanto a interface do utilizador como as mensagens de erro e os códigos de resposta da API.

- **Idiomas suportados:** A plataforma disponibiliza três idiomas — Português, Inglês e Espanhol — configuráveis pelo utilizador nas suas preferências de perfil;
- **Deteção automática:** Na plataforma Web, a biblioteca react-i18next [`i18next`] deteta automaticamente o idioma do navegador do utilizador, aplicando Português como idioma de *fallback* predefinido;
- **Ficheiros de localização:** As traduções estão organizadas em ficheiros JSON por idioma e por domínio (`common.json` para textos da interface e `api_codes.json` para mensagens de erro da API), garantindo a separação de contextos e a facilidade de manutenção;
- **Tradução offline na aplicação Mobile:** A aplicação móvel utiliza o serviço ML Kit Translation [`mlkit`] para suportar traduções mesmo sem conectividade à Internet, alinhado com a arquitetura *offline-first* adotada;
- **Saudações dinâmicas:** O sistema implementa uma lógica de saudação contextual que apresenta “*Bem-vindo!*” no primeiro acesso, “*Seja bem-vindo novamente*” após uma ausência superior a 15 dias e saudações baseadas na hora do dia (“*Bom dia!*”, “*Boa Tarde!*”, “*Boa Noite!*”) nas restantes interações, traduzidas para o idioma selecionado;
- **Tradução automática de conteúdos dinâmicos:** Para além das strings estáticas da interface (geridas por ficheiros de localização), todos os conteúdos provenientes da base

de dados — nomes de badges, descrições, requisitos, títulos de Learning Paths, Service Lines, Áreas e Níveis — são traduzidos automaticamente em tempo de execução para o idioma selecionado pelo utilizador. A API integra com a Google Cloud Translation API [**googletranslate**], com cache Redis de 7 dias para otimizar custos e latência, enquanto a aplicação Web utiliza um sistema de *batching* que agrega múltiplos textos num único pedido HTTP para minimizar chamadas à API. Na aplicação móvel, a tradução *offline* é assegurada pelo ML Kit Translation [**mlkit**].

3

Arquitetura de Informação e Modelo de Dados

Neste capítulo é apresentada a arquitetura de informação que sustenta a Plataforma de Badges da Softinsa. O modelo de dados foi desenhado em PostgreSQL 18 [**postgresql**] e compreende 44 tabelas organizadas em domínios funcionais distintos, interligados por chaves estrangeiras rigorosas e protegidos por restrições de integridade (*check constraints*) ao nível do SGBD. Para facilitar a análise, os diagramas são apresentados por domínio funcional, sendo cada secção acompanhada de uma descrição das tabelas envolvidas, das suas relações e das regras de negócio que motivaram as decisões de modelação.

3.1 Gestão de Utilizadores e Controlo de Acessos

O modelo de utilizadores segue um padrão de herança por tabela especializada (*Table-per-Subclass*). A tabela central `users` contém os atributos comuns a todos os perfis — nome completo, *email*, *hash* da *password*, papel (`user_role`), identificador público UUID (`user_guid`), idioma preferido e localização — enquanto cada papel do sistema possui uma tabela própria que estende a tabela base com atributos específicos:

- `administrators`: Contém a *flag* `is_super_admin` para distinguir administradores com privilégios totais;
- `consultants`: Armazena o consentimento RGPD (`gdpr_accepted`) e a biografia do consultor;
- `talent_managers`: Regista a biografia do Talent Manager, sem associação a Service Lines;
- `service_line_leaders`: Associa cada SLL à sua Service Line através de `service_line_id`, restringindo o seu âmbito de atuação.

A relação entre `users` e cada tabela de papel é de um-para-um, sendo a chave primária da tabela especializada simultaneamente chave estrangeira para `users`. O campo `user_role` na tabela base é validado por um *check constraint* que aceita exclusivamente os valores `Consultant`, `Talent Manager`, `Service Line Leader` e `Administrator`.

O suporte à autenticação é assegurado por duas tabelas auxiliares. A tabela `user_account_tokens` gere *tokens* de confirmação de conta e de recuperação de *password*, com tipo discriminado pelo campo `token_type` e validade temporal controlada por `expires_at`. A tabela `user_refresh_tokens` armazena *refresh tokens* JWT, incluindo uma *flag* `is_persistent` que suporta a funcionalidade de sessão persistente (“remember me”).

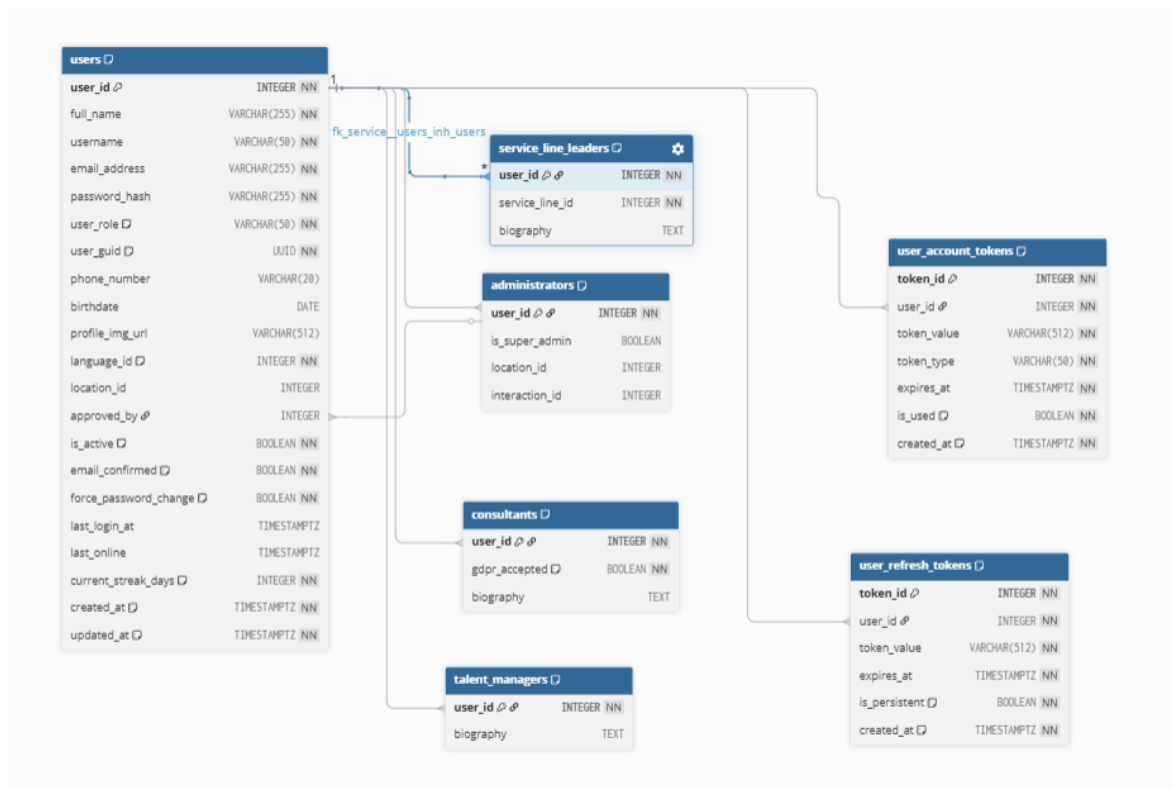


Figura 3.1: Modelo de dados — Gestão de utilizadores e controlo de acessos (RBAC).

3.2 Hierarquia de Conteúdos e Progressão

A organização dos conteúdos formativos segue uma hierarquia de cinco níveis, refletindo a estrutura organizacional da Softinsa. Cada nível é representado por uma tabela própria, com relações de um-para-muitos a percorrer toda a cadeia:

1. `learning_paths`: Nível de topo, representando os grandes itinerários de aprendizagem da empresa. Cada LP é identificado por um `path_slug` único e pode ser ativado ou desativado sem remoção dos dados;
2. `service_lines`: Agrupamento de competências técnicas dentro de um LP, associado

por `learning_path_id`. Cada SL possui igualmente um *slug* único para exposição segura na API;

3. `areas`: Subdivisões temáticas dentro de uma SL, com `service_line_id` como chave estrangeira;
4. `progression_stages`: Níveis de progressão dentro de uma área, ordenados pelo campo `stage_sequence`. Cada nível é classificado por um código de tipo através da tabela de referência `stage_codes`;
5. `badges`: Representação da credencial digital associada a cada nível de progressão. A relação com `progression_stages` é de um-para-um (restrição `UNIQUE` sobre `progression_stage_id`), garantindo que cada nível possui no máximo um badge. O campo `badge_type` distingue badges *Standard* de badges *Special*, e o campo `expiration_duration_days` permite configurar a validade temporal da credencial.

A tabela `stage_codes` funciona como tabela de referência para tipificar os níveis de progressão, permitindo a reutilização de códigos de nível (por exemplo, *Foundation*, *Associate*, *Professional*) de forma transversal a diferentes áreas. Todos os níveis da hierarquia incluem campos de auditoria (`created_by`, `updated_by`) que referenciam a tabela de administradores, rastreando quem criou e modificou cada entidade.

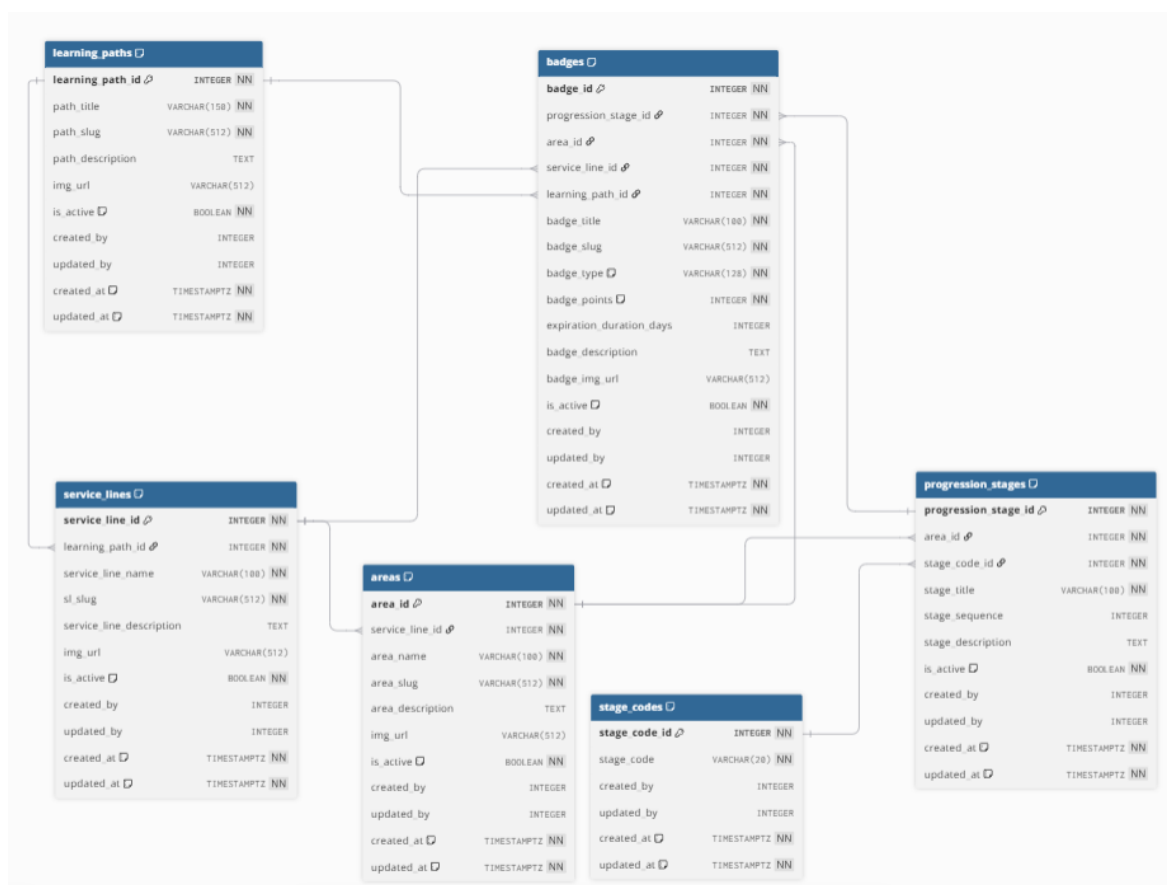


Figura 3.2: Modelo de dados — Hierarquia de conteúdos e progressão.

3.3 Ciclo de Vida dos Badges

O ciclo de vida de um badge compreende três domínios funcionais interligados: o *workflow* de candidaturas, a estrutura de requisitos e competências, e o sistema de gamificação por pontos.

3.3.1 Workflow de Candidaturas

A tabela `badge_applications` constitui o eixo central do processo de candidatura. Cada registo representa uma candidatura de um consultor a um badge específico, identificada externamente por um UUID (`application_guid`) gerado automaticamente pela extensão `pgcrypto`. O campo `application_state` controla o estado atual da candidatura através de um *check constraint* que restringe os valores a cinco estados: `Open`, `Submitted`, `In validation`, `Accepted` e `Rejected`. Uma restrição temporal adicional (`ckc_date_flow`) garante a coerência cronológica das transições, impedindo que a data de submissão seja anterior à de abertura ou que a data de fecho preceda a submissão.

Cada transição de estado é registada de forma imutável na tabela `application_validation_logs`, que funciona como trilha de auditoria do processo. Cada entrada regista o identificador da candidatura, o utilizador que efetuou a ação (`user_id`), a função do validador (`validator_function`), a ação executada (`validator_action`), comentários opcionais e o *timestamp* da validação. Este registo é estritamente aditivo — as entradas nunca são alteradas ou removidas.

Quando uma candidatura atinge o estado `Accepted`, é criado um registo na tabela `awarded_badges`, que materializa a atribuição efetiva da credencial. Esta tabela armazena a data de atribuição, a data de expiração (calculada a partir do `expiration_duration_days` do badge), um *snapshot* dos pontos obtidos, e a ligação pública de verificação (`public_verification_link`) — um URL único que permite a validação externa da credencial. As *flags* `is_published` e `is_featured` controlam a visibilidade do badge na galeria pública do consultor.

A tabela `certificates` complementa a candidatura, permitindo o armazenamento de metadados sobre os documentos de suporte (título, entidade emissora, data de emissão e URL do ficheiro no armazenamento externo).



Figura 3.3: Modelo de dados — Workflow de candidaturas a badges.

3.3.2 Requisitos e Competências

Cada badge possui um conjunto de requisitos definidos na tabela `badge_requirements`, numa relação de um-para-muitos. Cada requisito inclui um título, uma descrição, uma sequência de ordenação (`requirement_sequence`) e um valor de pontos próprio (`badge_points`), permitindo que a pontuação total de um badge seja distribuída granularmente pelos seus requisitos individuais. Uma restrição *check* garante que os pontos são sempre não negativos.

O modelo inclui ainda um catálogo de competências transversais, materializado na tabela `skills`. A associação entre badges e competências é implementada através da tabela de junção `badge_skills` (relação muitos-para-muitos), permitindo que um mesmo *skill* seja

partilhado por vários badges e que cada badge agrupe múltiplas competências. De forma complementar, a tabela `consultants_selected_skills` regista as competências selecionadas por cada consultor como áreas de interesse, suportando funcionalidades de recomendação personalizada de badges.

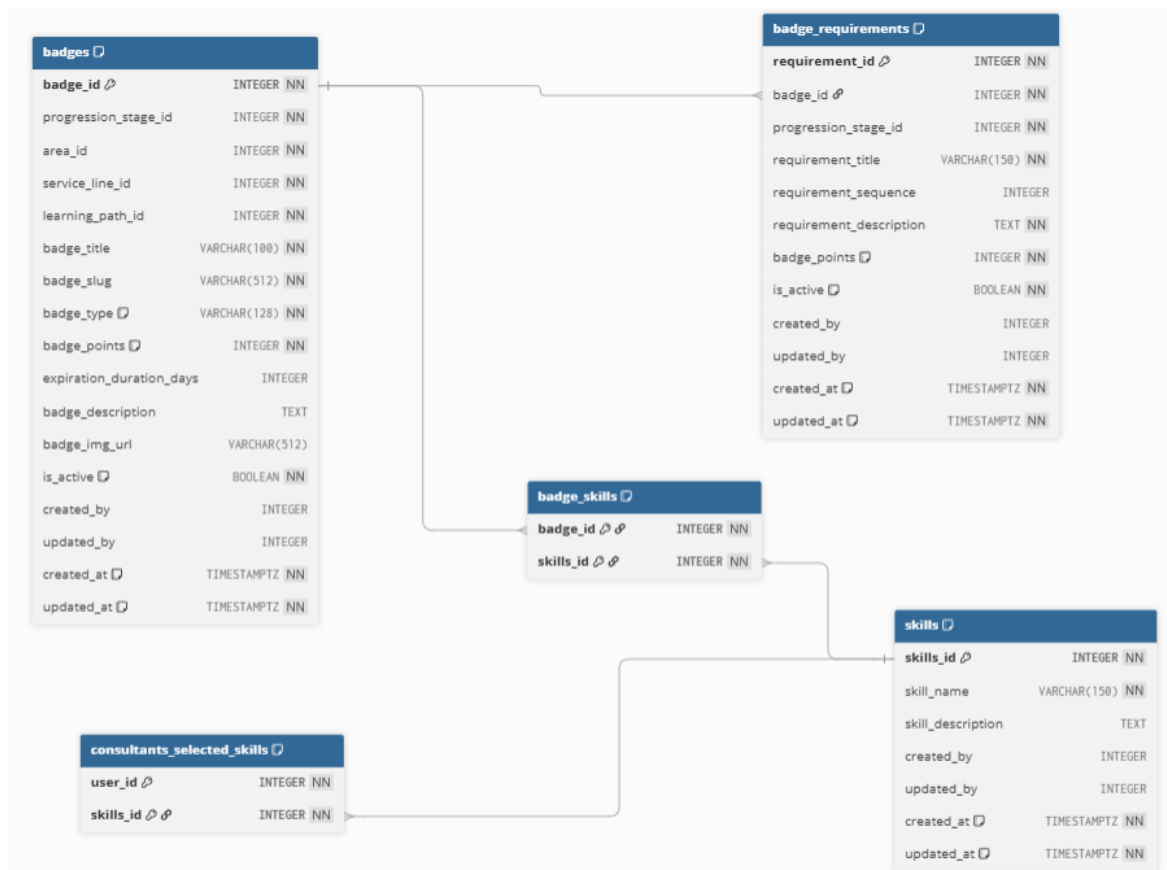


Figura 3.4: Modelo de dados — Requisitos de badges e competências.

3.3.3 Gamificação e Sistema de Pontos

O sistema de pontos é sustentado por um modelo de acumulação permanente, conforme exigido pelas regras de negócio: os pontos permanecem no registo do consultor mesmo que o badge subjacente expire. A tabela `points_history` implementa esta lógica como um registo de auditoria aditivo, onde cada entrada contém o consultor (`user_id`), a referência ao badge e/ou requisito que originou os pontos, o delta de pontos (`points_delta`) e uma justificação textual.

A pontuação é rastreada a dois níveis: ao nível do badge (campo `badge_points` na tabela `badges`) e ao nível de cada requisito individual (campo `badge_points` na tabela `badge_requirements`). Quando um badge é atribuído, o campo `points_snapshot` na tabela `awarded_badges` captura o total de pontos no momento da atribuição, preservando uma fotografia histórica independente de alterações futuras à configuração de pontos.

A Figura 3.5 ilustra a relação entre badges, candidaturas, badges atribuídos e o histórico

de pontos, evidenciando o fluxo de acumulação desde a definição do badge até ao registo permanente no perfil do consultor.

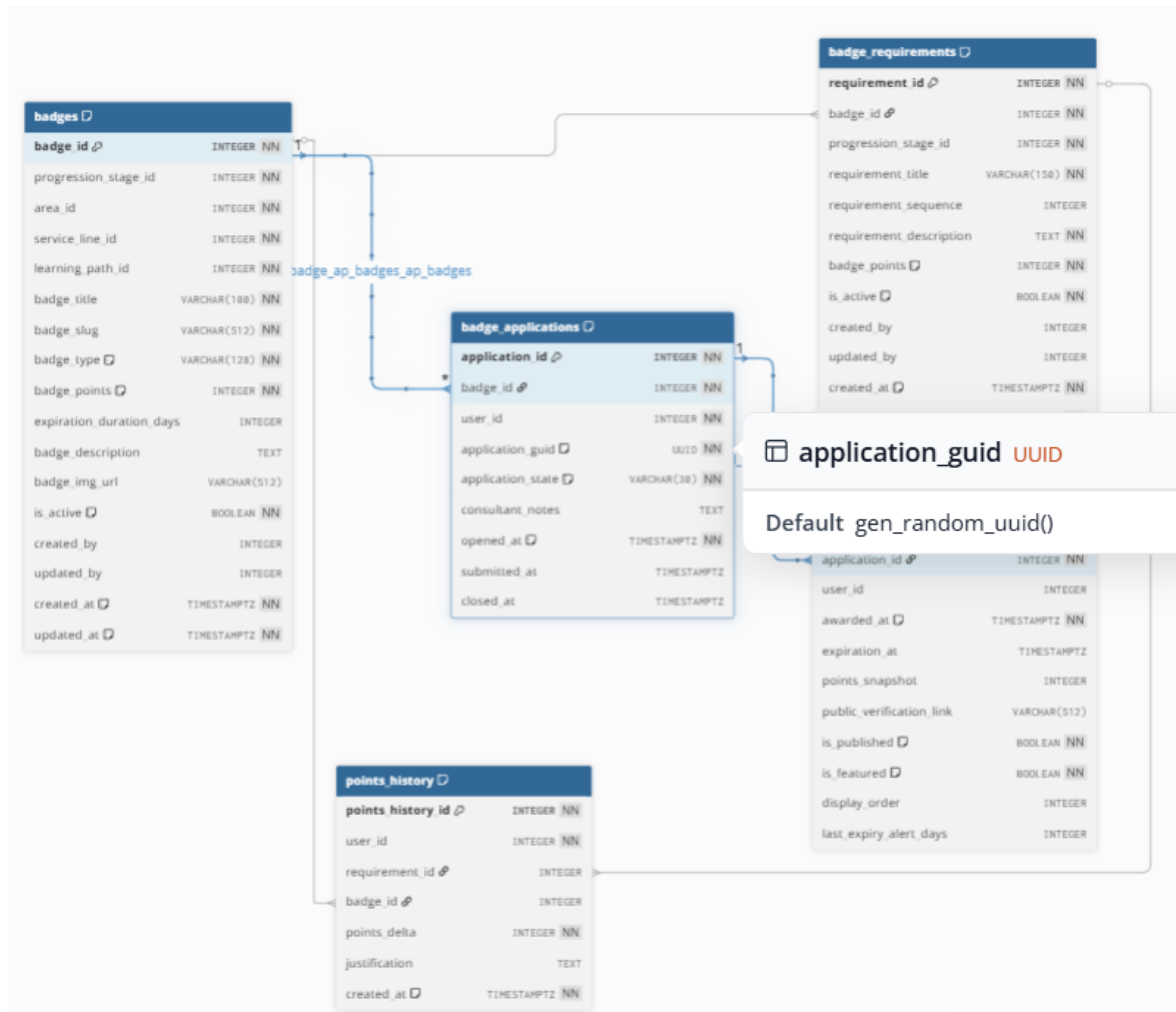


Figura 3.5: Modelo de dados — Sistema de gamificação e histórico de pontos.

3.4 Gestão Operacional e Conformidade

Para além do domínio central dos badges, o modelo de dados contempla três subsistemas de suporte à gestão operacional da plataforma: os acordos de nível de serviço (SLAs), o sistema de comunicação e avisos, e o módulo de conformidade com o RGPD.

3.4.1 Gestão de SLAs

O módulo de SLAs permite ao administrador definir acordos de nível de serviço que estabelecem tempos máximos de resposta para as equipas de validação. A tabela *slas* armazena cada definição, incluindo o nome, o tempo de resposta em horas (*response_time_hours*, validado por *check constraint* para valores positivos), as datas de vigência (*start_date*,

end_date), o perfil-alvo (target_profile) e uma *flag* is_global que indica se o SLA se aplica globalmente ou apenas a Service Lines específicas.

A associação entre SLAs e Service Lines é implementada pela tabela de junção sl_slas, que suporta a atribuição de múltiplos SLAs a uma mesma SL e vice-versa (relação muitos-para-muitos). Quando um SLA é violado, o sistema regista um alerta na tabela sla_breach_alerts, que referencia o SLA, a candidatura em causa (quando aplicável) e o utilizador notificado. Um índice único composto (idx_sla_breach_unique) previne a duplicação de alertas para a mesma combinação de SLA, candidatura e destinatário.



Figura 3.6: Modelo de dados — Gestão de SLAs e Service Lines.

3.4.2 Avisos e Comunicação do Sistema

O sistema de comunicação interna é centrado na tabela `system_announcements`, que suporta a criação de avisos globais com título, mensagem, tipo (Information, Warning, New Content ou Other), período de vigência (`starts_at`, `ends_at`) e estado ativo/inativo.

A segmentação da audiência é implementada por duas tabelas de junção complementares. A tabela `announc_roles` permite direcionar um aviso a papéis específicos (por exemplo, apenas a consultores e TMs), utilizando uma chave composta (`announcement_id`, `role_name`) com validação dos nomes de papel por *check constraint*. A tabela `announc_sl` permite restringir a visibilidade a Service Lines específicas, utilizando igualmente uma chave composta (`announcement_id`, `service_line_id`). A *flag* `is_global` na tabela principal permite que um aviso ultrapasse ambos os filtros, sendo apresentado a todos os utilizadores independentemente do papel ou SL.

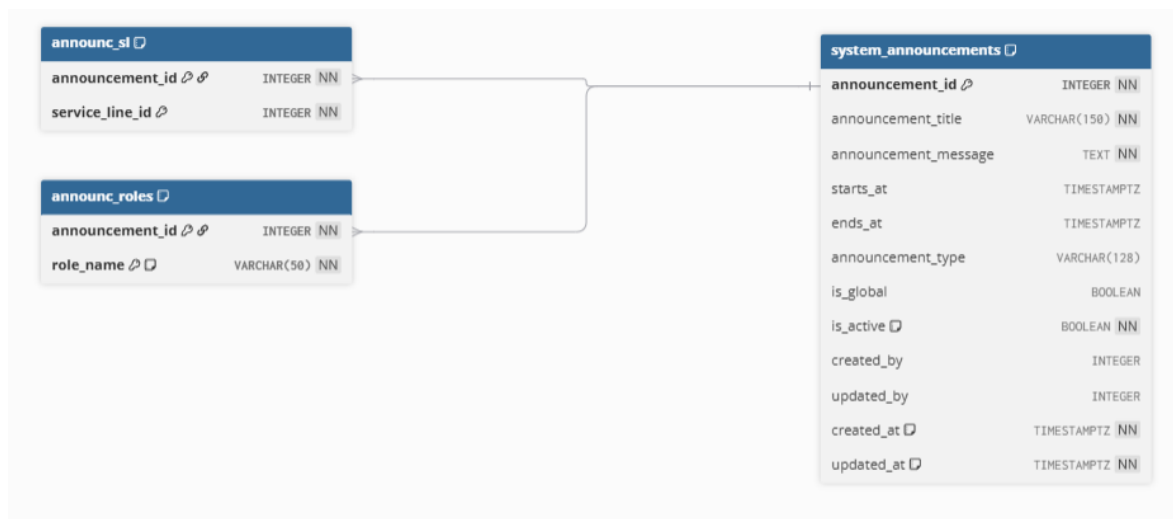


Figura 3.7: Modelo de dados — Sistema de avisos e comunicação.

3.4.3 RGPD e Gestão de Consentimentos

A conformidade com o Regulamento Geral sobre a Proteção de Dados [rgpd] é assegurada por um módulo dedicado composto por duas tabelas. A tabela `gdpr_policies` armazena as políticas de privacidade, termos de utilização e políticas de *cookies*, tipificadas pelo campo `policy_type` (validado por *check constraint* para os valores Privacy, Terms e Cookies). Cada política é versionada pelo campo `version`, com uma restrição de unicidade composta (`policy_type`, `version`) que impede versões duplicadas. A *flag* `is_mandatory` sinaliza as políticas cuja aceitação é obrigatória para a utilização da plataforma.

A tabela `gdpr_consent_history` implementa uma trilha de auditoria imutável para todos os consentimentos. Cada registo armazena o utilizador, a política aceite ou revogada (campo `action` com valores ACCEPTED ou REVOKED), o endereço IP de origem (`ip_address`), o *user agent* do navegador e o *timestamp* da ação. Este registo nunca é alterado nem eliminado,

garantindo rastreabilidade completa para fins de auditoria.

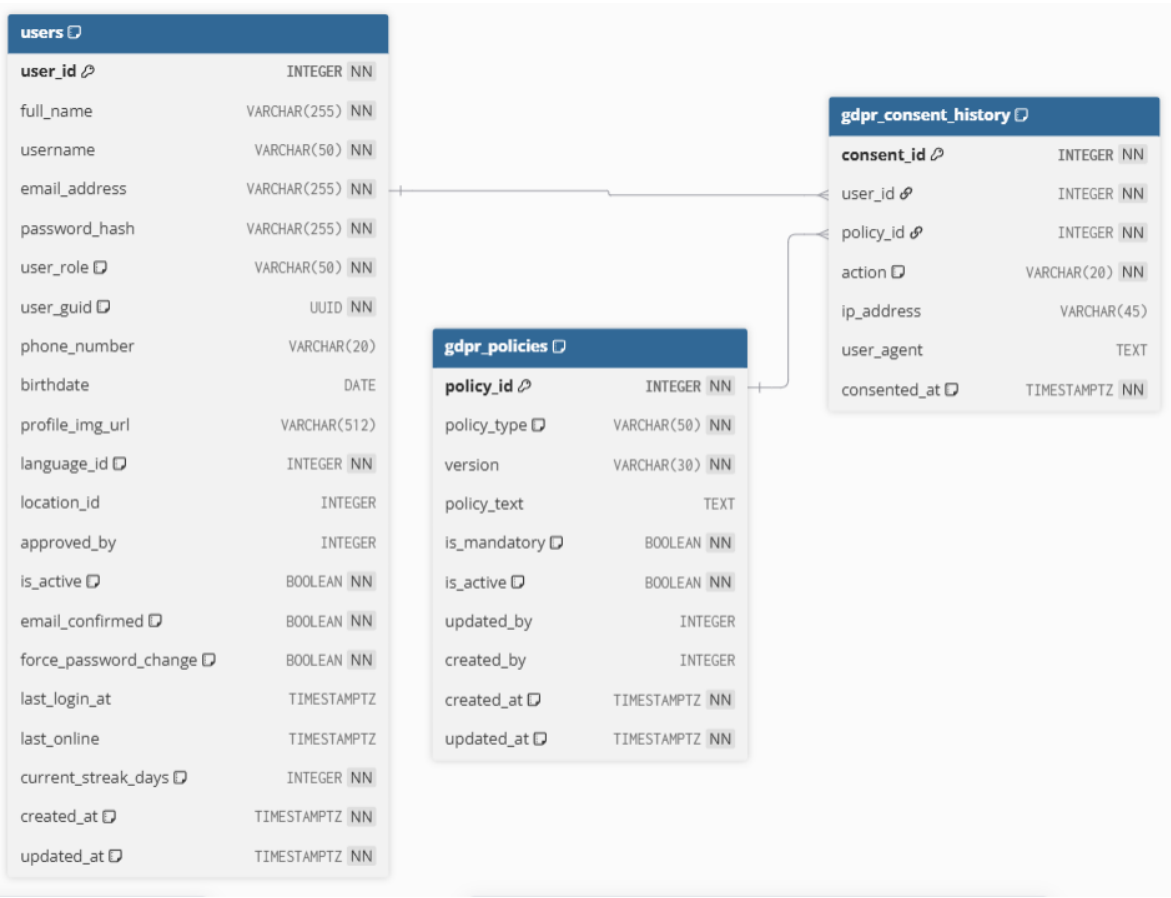


Figura 3.8: Modelo de dados — RGD e gestão de consentimentos.

4

Prototipagem e Design de Interface (Figma)

A conceção visual da plataforma partiu de um Protótipo de Alta Fidelidade (PAF) desenvolvido em Figma [figma], cobrindo os fluxos dos quatro perfis de utilizador — Consultor, Talent Manager, Service Line Leader e Administrador — bem como a aplicação móvel do Consultor. Este capítulo apresenta, de forma breve, uma seleção representativa desses ecrãs de alta fidelidade, organizada por perfil de utilizador.

4.1 Demonstração de Ecrãs

4.1.1 Consultor

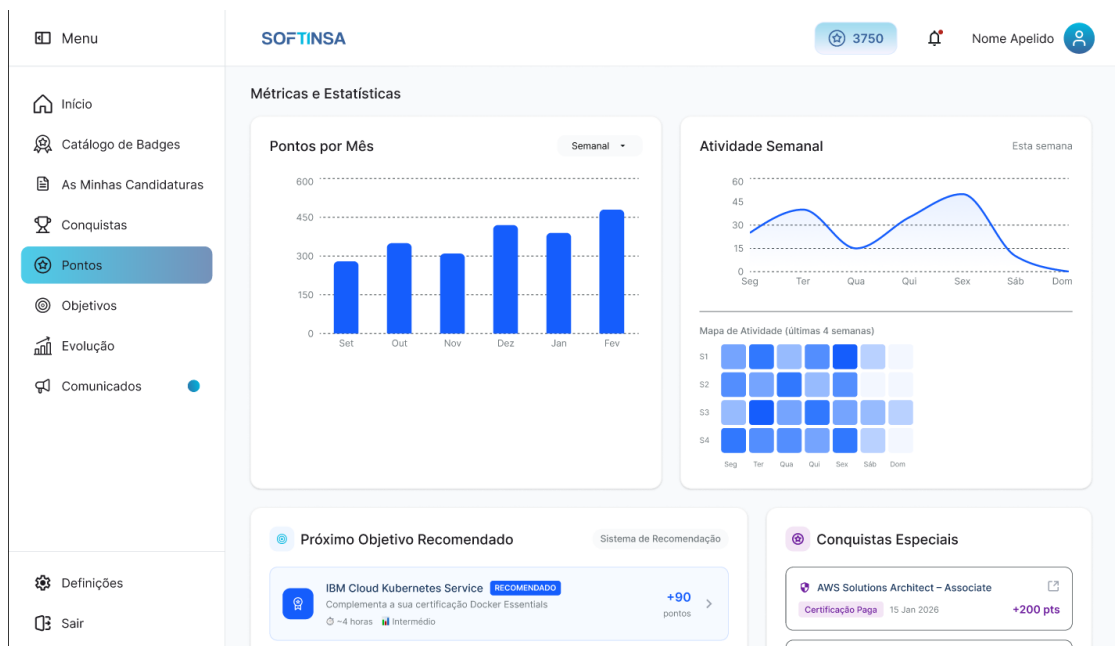


Figura 4.1: Ecrã de Pontos e gamificação do Consultor (gráficos de pontos, atividade e histórico).

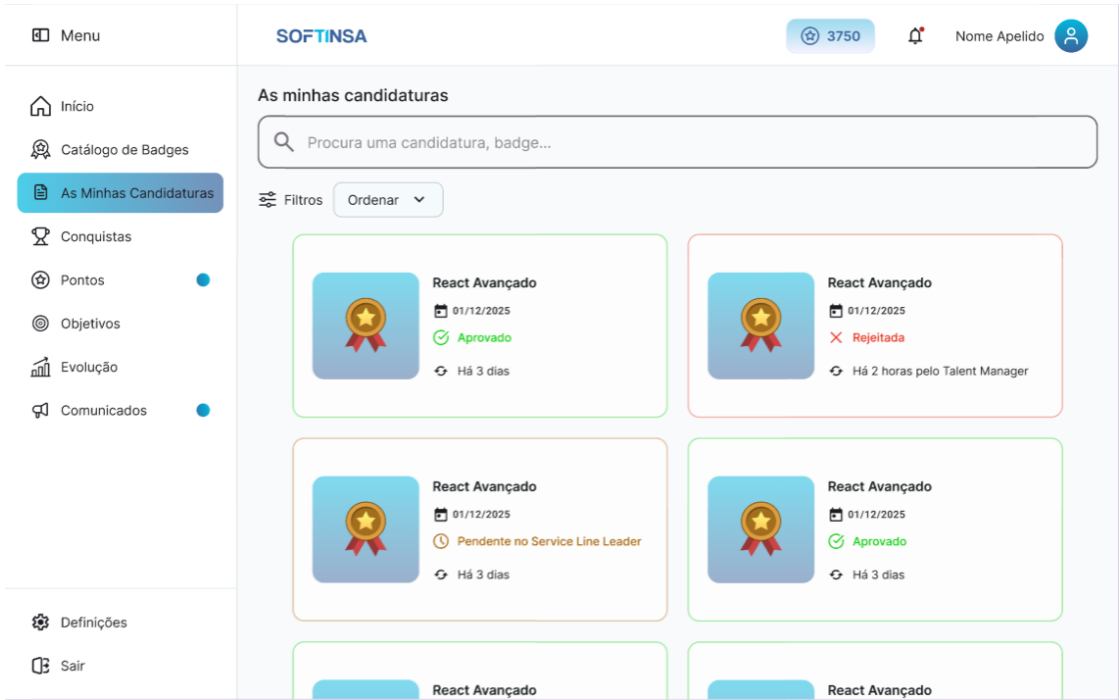


Figura 4.2: Lista de candidaturas do Consultor (“As Minhas Candidaturas”).

4.1.2 Talent Manager

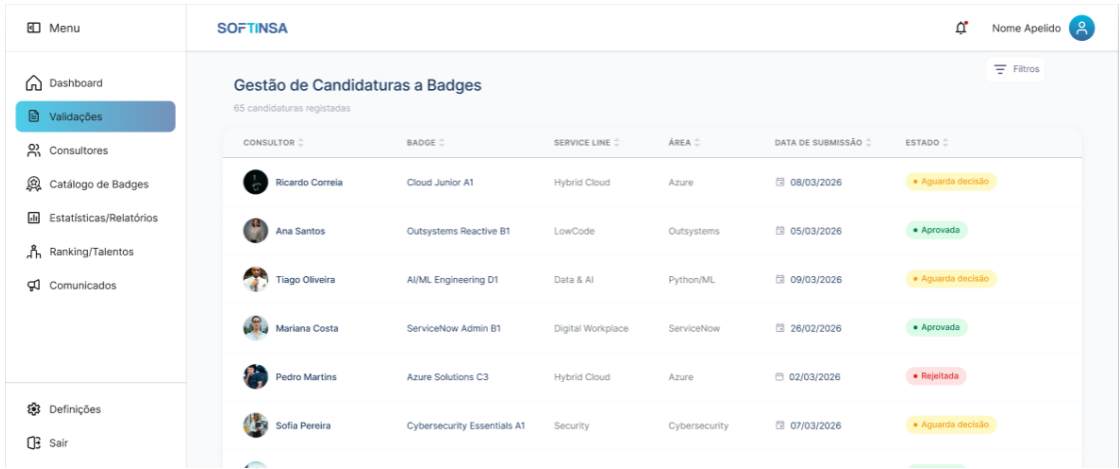


Figura 4.3: Ecrã do Talent Manager para a triagem e revisão de candidaturas.

4.1.3 Service Line Leader

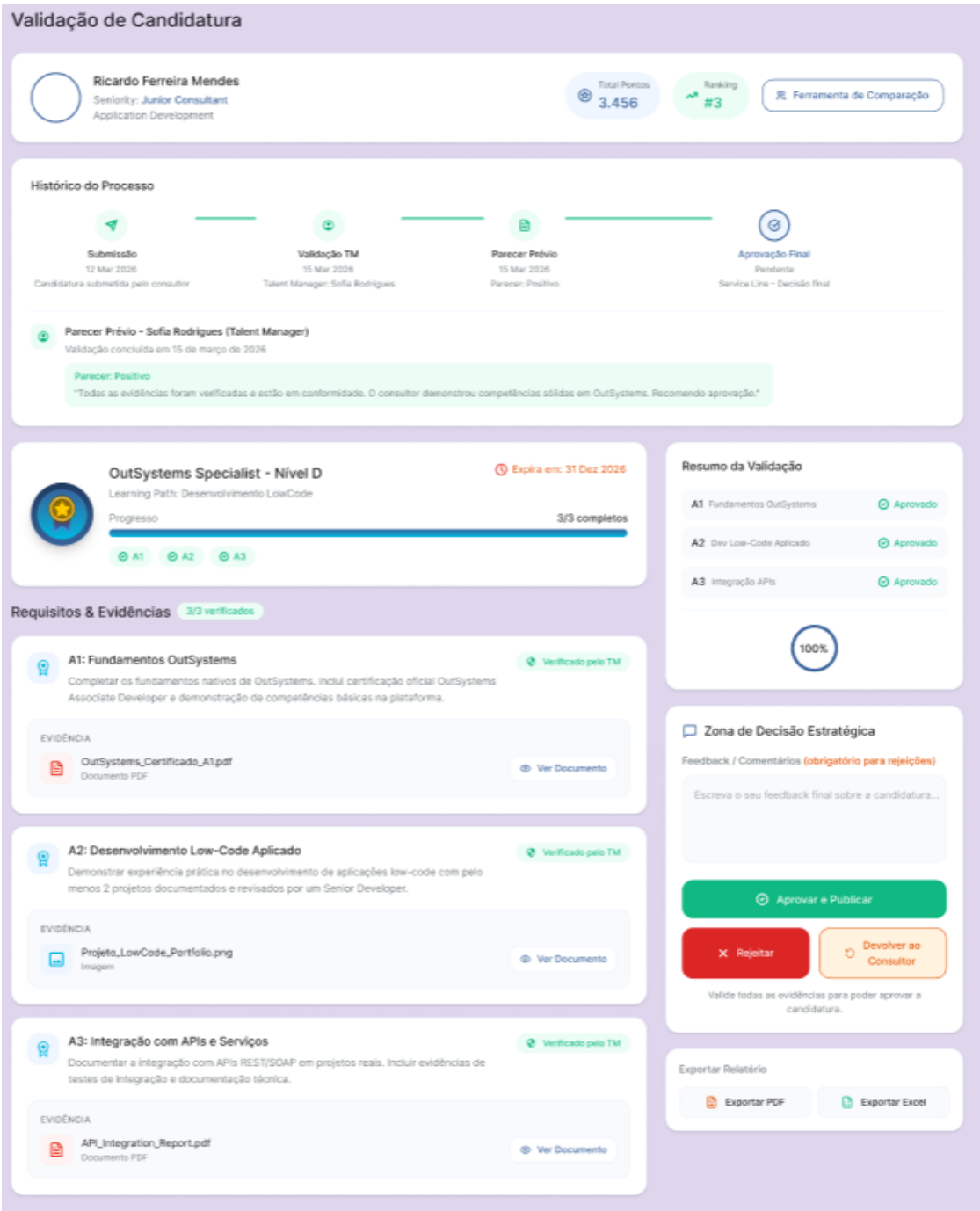


Figura 4.4: Ecrã de validação das evidências de uma candidatura (Service Line Leader).

4.1.4 Administrador

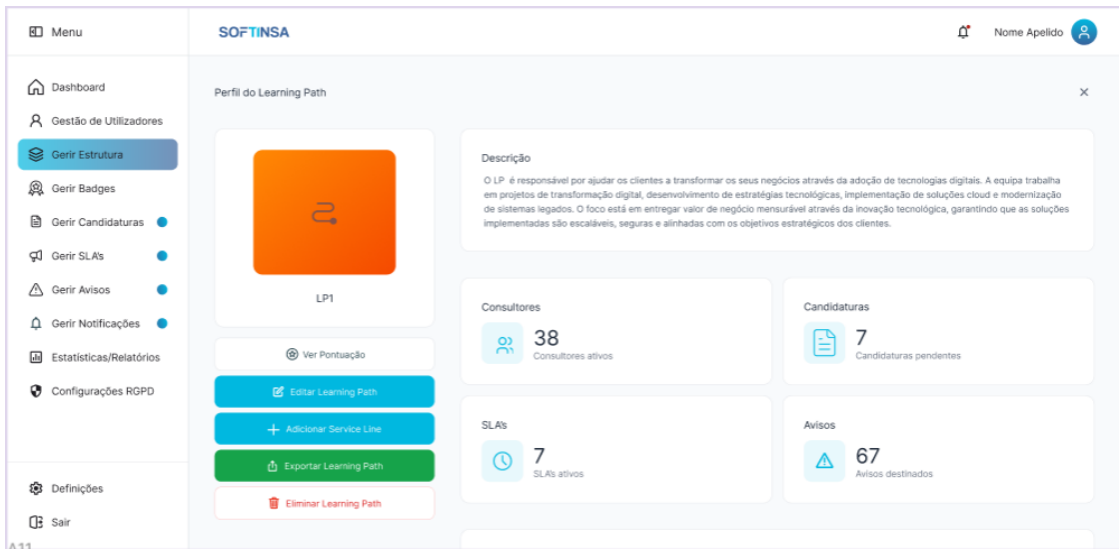


Figura 4.5: Gestão de Learning Paths e da hierarquia de conteúdos (Administrador).

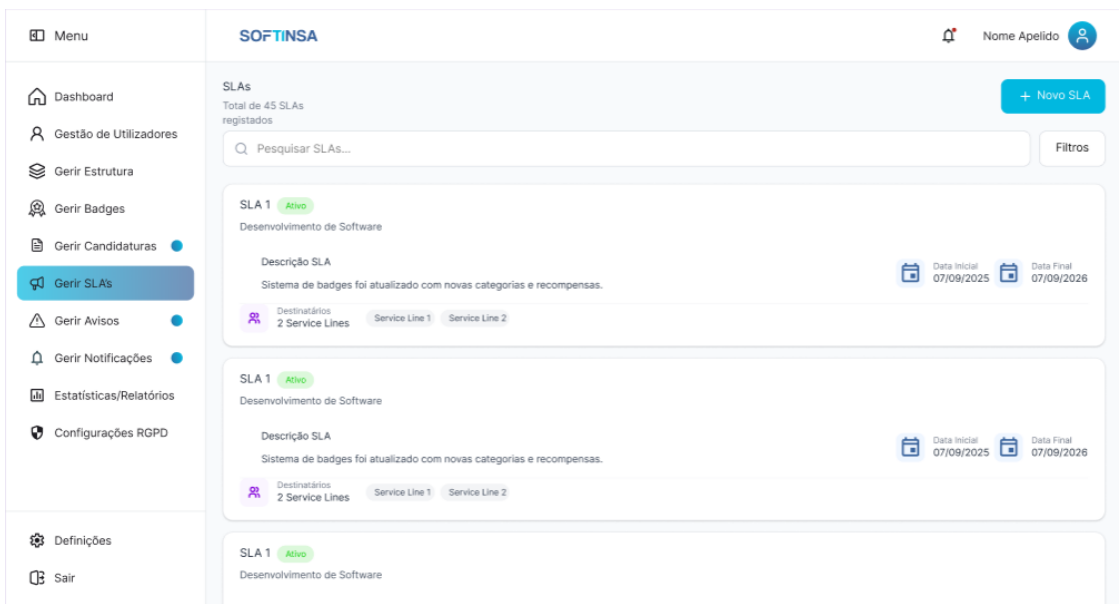


Figura 4.6: Configuração de SLAs da plataforma (Administrador).

4.1.5 Aplicação Móvel

Os fluxos do Consultor foram igualmente prototipados para a aplicação móvel, seguindo a mesma identidade visual da versão Web.

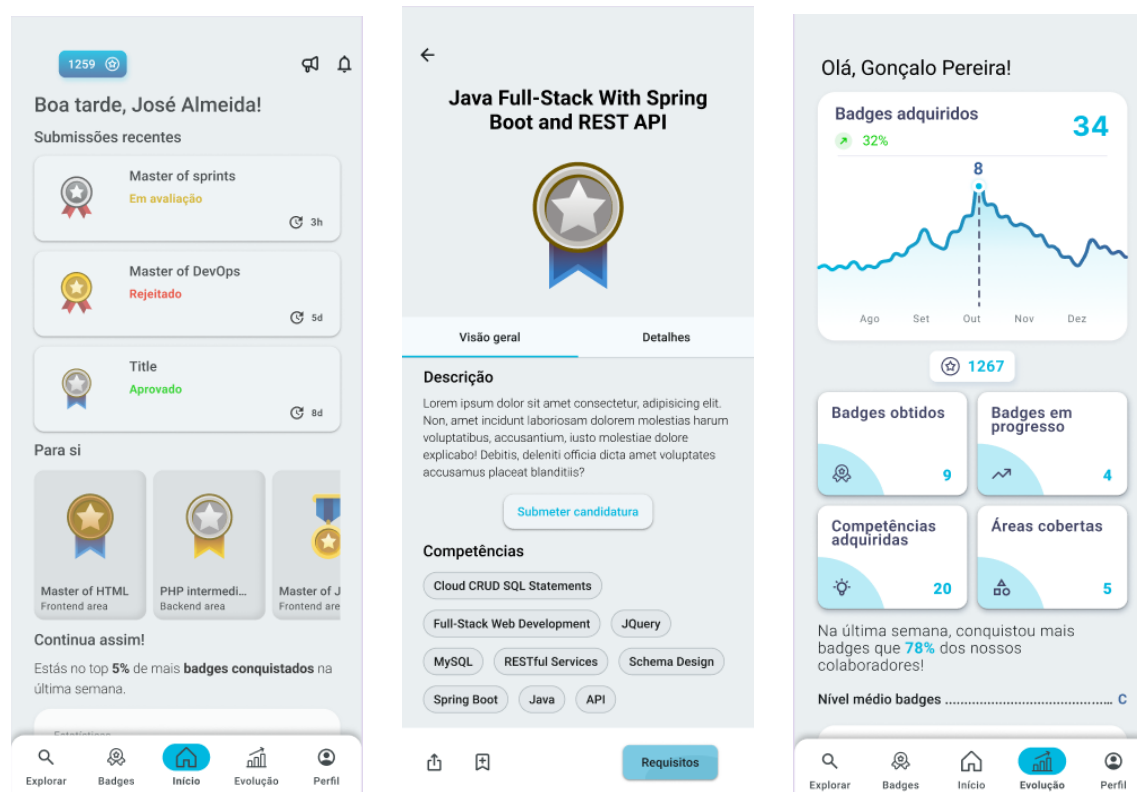


Figura 4.7: Aplicação móvel do Consultor: dashboard, detalhe de badge e ecrã de evolução.

4.2 Validação com a Softinsa

Estes protótipos — e os restantes ecrãs que compõem o Protótipo de Alta Fidelidade — foram apresentados à Softinsa, que reagiu de forma positiva e validou os conceitos propostos. Ao longo da implementação procurámos segui-los o mais fielmente possível, introduzindo apenas os ajustes necessários para assegurar a responsividade nos vários tamanhos de ecrã e para acomodar a totalidade dos requisitos do sistema.

5

Workflow de Aprovação e Ciclo de Vida do Pedido

5.1 Submissão (Consultor)

5.2 Triagem e Validação de Evidências (Talent Manager)

5.3 Homologação Final (Service Line Leader)

6

Funcionalidades Transversais Implementadas

6.1 Mecânicas de Gamificação

6.2 Módulo de Estatísticas e Reporting

7

Extras e Funcionalidades Bónus

7.1 Sistema de Saudações Inteligente

O sistema de saudações inteligente constitui uma funcionalidade de personalização da experiência do utilizador que adapta a mensagem de boas-vindas ao contexto específico de cada acesso à plataforma. A lógica implementada distingue três cenários mutuamente exclusivos, avaliados por ordem de prioridade conforme descrito na Tabela ??.

Cenário	Condição	pt-PT	en-GB	es-ES
Primeiro acesso	<code>last_login</code> é NULL	Bem-vindo!	Welcome!	¡Bienvenido!
Regresso > 15 dias	Diferença entre <code>last_online</code> e a data atual superior a 15 dias	Seja bem-vindo novamente	Welcome back	Bienvenido de nuevo
Manhã	Hora local $\geq 6h$ e $< 13h$	Bom dia!	Good morning!	¡Buenos días!
Tarde	Hora local $\geq 13h$ e $< 20h$	Boa tarde!	Good afternoon!	¡Buenas tardes!
Noite	Hora local $\geq 20h$ ou $< 6h$	Boa noite!	Good evening!	¡Buenas noches!

A implementação distribui-se entre o *backend* e o *frontend*. No lado do servidor, a tabela `users` contém três campos dedicados a este mecanismo: `last_login_at` (regista o *timestamp* do último *login*), `last_online` (atualizado em cada *login* e renovação de *token*) e `current_streak_days` (contabiliza os dias consecutivos de acesso). Durante o processo de autenticação, o controlador captura o valor prévio de `last_login_at` antes de o atualizar, determinando assim se se trata de um primeiro acesso (campo nulo) e devolvendo a *flag* `first_login` na resposta da API.

No lado do cliente, o componente `WelcomeCard` implementa a função `getGreeting()`, que avalia os três cenários por ordem de prioridade. A verificação do primeiro acesso e do regresso após ausência prolongada utiliza os dados recebidos do *backend* no momento da autenticação. A saudação por período do dia é calculada localmente com base na hora do dispositivo do utilizador. As mensagens são geridas pelo sistema de internacionalização `react-i18next` [**i18next**], com traduções definidas nos ficheiros de localização de cada idioma (`common.json`).

O componente `WelcomeCard` é integrado nos dashboards de todos os perfis de utilizador — Consultor, Talent Manager, Service Line Leader e Administrador — garantindo uma experiência personalizada e consistente independentemente do papel.

Complementarmente, o sistema mantém um registo de *streak* de acesso consecutivo. A função `getUpdatedStreak()` compara as datas UTC do último acesso e do acesso atual: se o acesso ocorre no dia seguinte ao anterior, o contador incrementa; caso exista um intervalo superior a um dia, o contador reinicia para um. Este valor é utilizado pela componente de gamificação para recompensar a assiduidade dos consultores na plataforma.

7.2 Mural de Avisos e Comunicação

O módulo de avisos e comunicação fornece aos perfis de gestão da plataforma um canal centralizado para a difusão de informações, alertas e novidades dirigidos a segmentos específicos da base de utilizadores. O sistema suporta a segmentação por papel, por Service Line e a difusão global, com controlo de período de vigência e estado ativo/inativo.

7.2.1 Modelo de Dados e Segmentação

A arquitetura do módulo assenta em três tabelas, conforme descrito na Secção 3.4.2 do Capítulo 3. A tabela principal `system_announcements` armazena o conteúdo do aviso — título (até 150 caracteres), mensagem (até 10 000 caracteres), tipo (`Information`, `Warning`, `New Content` ou `Other`), período de vigência (`starts_at`, `ends_at`) e estado de ativação. A segmentação da audiência é implementada por duas tabelas de junção: `announc_roles`, que direciona o aviso a papéis específicos, e `announc_sl`, que restringe a visibilidade a Service Lines concretas. A *flag* `is_global` permite ultrapassar ambos os filtros, tornando o

aviso visível a todos os utilizadores. Quando um aviso não é global, a validação Zod [**zod**] exige que seja especificado pelo menos um papel ou uma Service Line como destinatário.

Um índice composto sobre os campos `is_active`, `starts_at` e `ends_at` (`idx_announcements_active`) otimiza as consultas de avisos ativos dentro do período de vigência, que constitui o cenário de utilização mais frequente.

7.2.2 Lógica de Visibilidade e Controlo de Acesso

A API implementa regras de visibilidade diferenciadas consoante o perfil do utilizador autenticado. Os perfis de gestão (Administrador, TM e SLL) acedem a todos os avisos, incluindo os inativos, o que lhes permite gerir o ciclo de vida completo de cada publicação. Os consultores visualizam apenas os avisos ativos cujo período de vigência abranja a data atual, filtrados pela correspondência com o seu papel ou com as Service Lines associadas às suas áreas de interesse. A verificação de pertença à Service Line é resolvida através da cadeia `consultant_areas` → `areas` → `service_lines`, garantindo que o consultor apenas visualiza avisos dirigidos à sua área.

No que respeita à gestão, os três perfis de liderança (Administrador, TM e SLL) podem criar avisos, mas apenas o Administrador pode editar ou desativar avisos criados por outros utilizadores. A desativação é implementada como *soft delete*, atualizando a *flag* `is_active` para FALSE sem remoção física do registo, preservando o histórico de comunicações.

7.2.3 Interface de Gestão e Visualização

O *frontend* apresenta duas experiências distintas, selecionadas automaticamente pelo componente `AnnouncementsPage` com base no papel do utilizador autenticado. Os perfis de gestão acedem a uma interface tabular com funcionalidades completas de CRUD, incluindo filtros por título, tipo e estado, paginação e um modal de criação/edição (`AnnouncementFormModal`) que permite configurar o título, a mensagem, o tipo, o período de vigência, a *flag* de globalidade e a seleção de papéis e Service Lines alvo. O tipo de cada aviso é apresentado com um *badge* cromático diferenciado, e cada linha da tabela exhibe o criador, o âmbito de destinatários e o estado ativo/inativo com indicador visual.

Os consultores acedem a uma vista em grelha de cartões com pré-visualização da mensagem (limitada a três linhas), pesquisa por título e modal de detalhe para consulta integral, sem funcionalidades de edição.

7.2.4 Otimização por Cache Dinâmico

A listagem de avisos beneficia de *cache* Redis [**redis**] com cálculo dinâmico do TTL. Ao armazenar uma resposta em *cache*, o sistema consulta as datas de início e fim de todos os avisos ativos para determinar a próxima alteração de visibilidade, definindo o TTL como o

mínimo entre o tempo até essa alteração e 7 200 segundos (2 horas). Esta estratégia garante que o *cache* é automaticamente invalidado quando um aviso entra ou sai do período de vigência, sem necessidade de invalidação manual. Adicionalmente, qualquer operação de escrita (criação, atualização ou desativação) invalida imediatamente todas as entradas de *cache* do módulo por prefixo.

7.2.5 Notificações Automáticas

A criação de um aviso desencadeia automaticamente a distribuição de notificações *in-app* aos utilizadores abrangidos através da função `dispatchAnnouncementNotifications()`. O processo de resolução de destinatários segue a mesma lógica de segmentação: avisos globais geram notificações para todos os utilizadores ativos; avisos dirigidos a papéis específicos notificam apenas os utilizadores com o papel correspondente; avisos dirigidos a Service Lines notificam os consultores com áreas associadas a essas SLs. Esta operação é executada em modo *fire-and-forget*, sem bloquear a resposta ao utilizador que criou o aviso.

7.3 Alertas de SLA e Notificações

O sistema de acordos de nível de serviço (SLAs) permite ao administrador definir tempos máximos de resposta para os processos de validação de candidaturas, monitorizando automaticamente o seu cumprimento e alertando os responsáveis quando estes são ultrapassados. Este módulo constitui uma das funcionalidades bônus mais complexas da plataforma, integrando definição de regras, monitorização automatizada, notificação multicanal e preferências configuráveis.

7.3.1 Definição e Configuração de SLAs

Cada SLA é definido na tabela `slas` com os seguintes parâmetros: nome descritivo, tempo de resposta em horas (`response_time_hours`, com *check constraint* para valores positivos), período de vigência (`start_date`, `end_date`), perfil-alvo (`target_profile`) e âmbito de aplicação. O campo `is_global` determina se o SLA se aplica a todas as Service Lines ou apenas às especificadas na tabela de junção `sl_slas`. O administrador gere os SLAs através de uma página dedicada no *frontend*, com formulários de criação e edição que incluem a seleção de Service Lines (quando o SLA não é global), validação de campos e controlo de estado ativo/inativo.

A plataforma suporta dois tipos distintos de SLAs:

- **SLAs de candidatura:** Medem o tempo de resposta dos validadores (TM ou SLL) às candidaturas que aguardam a sua intervenção. O tempo é calculado desde a submissão ou a passagem ao estado `In validation` até ao momento atual;

- **SLAs autónomos (*standalone*):** Representam prazos genéricos definidos pelo administrador, sem associação direta a candidaturas específicas, utilizados para monitorizar *deadlines* operacionais ou prazos de formação.

7.3.2 Monitorização Automatizada

A deteção de incumprimentos é realizada por dois *workers* independentes, ambos executados a cada 15 minutos através de node-cron [**nodecron**]:

- **`sla.worker`:** Executa uma consulta SQL que identifica candidaturas nos estados Submitted ou In validation cujo tempo de permanência excede o `response_time_hours` do SLA aplicável. A consulta cruza os dados da candidatura com os SLAs ativos dentro do período de vigência, verificando a correspondência do perfil-alvo com o estado da candidatura e a abrangência por Service Line;
- **`custom_sla.worker`:** Monitoriza SLAs autónomos cuja data de fim (`end_date`) foi ultrapassada, identificando os utilizadores-alvo com base no perfil e, quando aplicável, na associação a Service Lines específicas.

Ambos os *workers* implementam um mecanismo de prevenção de duplicados: antes de gerar um alerta, verificam na tabela `sla_breach_alerts` se a combinação (SLA, candidatura, utilizador) já foi notificada. Um índice único composto sobre estes três campos garante a unicidade a nível da base de dados. Os alertas são registos estritamente aditivos — nunca alterados nem removidos — formando uma trilha de auditoria dos incumprimentos detetados.

7.3.3 Notificações Multicanal

Os alertas de incumprimento são entregues através de três canais complementares, configuráveis de forma independente:

- **Notificações *in-app*:** Cada alerta gera um registo na tabela `notifications`, entregue em tempo real ao *frontend* através de WebSocket (Socket.IO [**socketio**]). O componente `NotificationIcon` na barra superior apresenta um indicador com a contagem de notificações não lidas, permitindo ao utilizador aceder ao centro de notificações com filtros por tipo e estado de leitura;
- **Notificações *push*:** Para os utilizadores da aplicação móvel, os alertas são enviados através do Firebase Cloud Messaging [**firebase**]. O sistema gere os *tokens* FCM dos dispositivos registados na tabela `device_tokens`, desativando automaticamente os *tokens* obsoletos após falhas de envio;

- **Notificações por e-mail:** O serviço de e-mail (Nodemailer [**nodemailer**] com transporte Gmail) disponibiliza *templates* HTML localizados nos três idiomas da plataforma. Os *templates* de alerta de SLA incluem o nome do candidato, o badge em causa, as horas excedidas e o perfil responsável, com botão de ação direto para aceder à candidatura.

7.3.4 Hierarquia de Preferências

O sistema implementa uma hierarquia de preferências a dois níveis que permite equilibrar o controlo centralizado com a personalização individual:

1. **Preferências globais** (`notification_preferences`): Definidas pelo administrador para cada tipo de notificação (incluindo `SLA_BREACH`, `APPLICATION_SUBMITTED`, `BADGE_EXPIRING_SOON`, entre outros), estabelecem os valores predefinidos para os campos `is_enabled`, `send_email` e `send_push`;
2. **Preferências do utilizador** (`user_notification_preferences`): Cada utilizador pode sobrepor os valores globais para qualquer tipo de notificação. Os campos com valor `NULL` herdam automaticamente o valor da preferência global correspondente.

A resolução de preferências é efetuada pela função `resolvePreferences()`, que consulta primeiro a preferência global e aplica as sobreposições do utilizador. Na ausência de qualquer configuração, todos os canais são considerados ativos por omissão, garantindo que nenhum alerta crítico é silenciado inadvertidamente. No *frontend*, o administrador dispõe de uma página dedicada para configurar as preferências globais, enquanto cada utilizador pode personalizar as suas preferências individuais através do componente `NotificationPreferences`.

8

Implementação Técnica de Funcionalidades

Este capítulo apresenta as decisões técnicas e os mecanismos transversais que sustentam o funcionamento do ecossistema, abrangendo tanto a API REST como a aplicação web. O foco incide sobre a infraestrutura de segurança, desempenho, comunicação e resiliência que permeia todas as funcionalidades descritas nos capítulos anteriores.

8.1 Segurança da Camada HTTP

A segurança da API é implementada como uma cadeia de *middleware* configurada no ficheiro `app.js`, que cada pedido HTTP atravessa sequencialmente antes de atingir qualquer lógica aplicacional. Esta cadeia foi desenhada para mitigar as ameaças mais comuns identificadas pelo OWASP Top Ten [[owasp](#)].

O Helmet [[helmet](#)] abre a cadeia, injetando automaticamente um conjunto de cabeçalhos HTTP de segurança em todas as respostas: a `Content-Security-Policy` restringe as fontes de conteúdo executável, o `X-Frame-Options` impede o carregamento da API em *iframes* para prevenir *clickjacking*, e o `X-Content-Type-Options` com o valor `nosniff` impede o *browser* de reinterpretar o tipo MIME das respostas.

A configuração de CORS restringe as origens aceites exclusivamente aos domínios da aplicação cliente, definidos nas variáveis de ambiente `APP_URL` e `WEB_APP_URL`, com suporte a credenciais por *cookies* e sem utilização de *wildcard*. Pedidos de domínios não autorizados são rejeitados antes de atingirem qualquer rota.

O *parsing* do corpo dos pedidos aceita JSON e dados URL-encoded com um limite rigoroso de 100 KB, protegendo contra ataques de negação de serviço por *payloads* excessivos. Os ficheiros de evidência, que podem atingir 10 MB, são carregados diretamente para o Supabase [[supabase](#)] pelo *browser*, sem transitar pelo corpo dos pedidos da API.

A proteção contra ataques de força bruta é implementada através do `express-rate-limit`,

com configurações calibradas por *endpoint* para equilibrar usabilidade e segurança:

- **Login:** 10 tentativas por janela de 15 minutos;
- **Registo e recuperação de password:** 5 tentativas por hora;
- **Reset e alteração de password:** 10 tentativas por janela de 15 minutos;
- **Refresh de token:** 100 pedidos por janela de 15 minutos, valor mais permissivo para acomodar o *polling* automático da SPA;
- **Reenvio de confirmação:** 5 tentativas por hora.

Estas configurações tornam inviáveis os ataques automatizados de enumeração de credenciais, mantendo a utilização normal da plataforma sem fricção.

A prevenção de Cross-Site Scripting (XSS) é assegurada pela biblioteca `sanitize-html`, que filtra todos os campos de texto livre antes do armazenamento, removendo *tags* HTML, atributos perigosos e *scripts* embutidos. Complementarmente, o `leo-profanity` com dicionários carregados para Português e Espanhol filtra palavras inadequadas em campos como nomes, biografias e comentários.

8.2 Autenticação e Gestão de Sessão

O sistema de autenticação implementa um fluxo completo baseado em JSON Web Tokens [**jwt**], concebido para equilibrar segurança, usabilidade e resiliência a ataques comuns.

8.2.1 Registo e Hashing de Credenciais

O processo de registo inicia-se com a submissão dos dados do utilizador, onde a password é imediatamente sujeita a *hashing* com `bcryptjs` [**bcrypt**] utilizando 12 *salt rounds* ($2^{12} = 4096$ iterações). Este valor foi selecionado para equilibrar resistência a ataques de dicionário com tempo de resposta aceitável. A criação do utilizador e do registo específico do papel (Consultor, Talent Manager ou Service Line Leader) ocorre numa transação atómica — se qualquer passo falhar, toda a operação é revertida. Após a criação, é enviado um email de confirmação contendo um token único com validade de 8 horas; o token é armazenado na base de dados sob forma de *hash* SHA-256, nunca em texto simples.

8.2.2 Fluxo de Login e Tokens

O *login* valida as credenciais via comparação `bcrypt` e, em caso de sucesso, gera dois tokens complementares. O *access token* JWT, com duração configurável (15 minutos por omissão), contém o identificador do utilizador, o seu papel e a *flag* de mudança forçada de

password (fpc), sendo armazenado exclusivamente em memória no lado do cliente — nunca em `localStorage` ou `sessionStorage`, mitigando o risco de roubo por XSS. O *refresh token*, de longa duração, é armazenado na base de dados sob forma de *hash* e enviado ao cliente como *cookie* HTTP-only com atributo `SameSite` configurado como `None` em produção (para suportar comunicação *cross-domain*) e `Lax` em desenvolvimento, com o atributo `Secure` ativado em produção.

Quando o *access token* expira, o cliente envia automaticamente o *refresh token* para obter um novo *access token* sem necessidade de re-autenticação. O *logout* revoga o *refresh token* na base de dados, impedindo futuras renovações.

8.2.3 Modos de Sessão e Monitorização de Atividade

A aplicação web implementa dois modos de sessão distintos, seleccionáveis pelo utilizador na interface de *login*. A sessão persistente (“Lembrar-me”) mantém uma janela de atividade de 5 minutos para renovação do token, sem desconexão forçada por inatividade, enquanto a sessão efémera utiliza uma janela de 10 minutos, após a qual o utilizador é desconectado automaticamente. A monitorização de atividade é realizada no *frontend* através de *event listeners* nos eventos *mousemove*, *click* e *keydown*, com *debounce* de 10 segundos para evitar processamento excessivo. Um verificador periódico executa a cada 30 segundos: se o tempo restante do *access token* for inferior a 60 segundos e existir atividade recente dentro da janela configurada, o *refresh* é acionado proativamente; caso contrário, a sessão é encerrada por inatividade.

A deduplicação de chamadas de *refresh* é garantida pela memoização da Promise: quando múltiplos pedidos simultâneos recebem HTTP 401, todos aguardam a mesma Promise de *refresh* em vez de dispararem chamadas redundantes.

8.2.4 Mudança Forçada de Password

Quando o campo `force_password_change` está ativo no registo do utilizador, o *middleware* de autenticação bloqueia todos os *endpoints* exceto o de alteração de password, devolvendo HTTP 403. No lado do cliente, a guarda de rota `ProtectedRoute` deteta a *flag* `fpc` no token JWT e redireciona o utilizador para a página de alteração de password, impedindo qualquer outra interação até que a mudança seja efetuada.

8.3 Controlo de Acesso Baseado em Papéis

O controlo de acesso opera em três níveis complementares: *middleware* de rota no *backend*, guardas de rota no *frontend* e filtragem de dados ao nível da *query SQL*.

8.3.1 Middleware de Autorização

O *middleware* `loginRequired` verifica a presença e validade do JWT no cabeçalho de autorização, extraindo os dados do utilizador para `req.user`. O *middleware* `checkRole` é uma função fábrica que recebe uma lista de papéis permitidos e gera um verificador, originando dois atalhos frequentes: `isAdmin` (apenas Administrador) e `leadership` (Service Line Leader, Talent Manager ou Administrador). O `optionalAuth` tenta decodificar o JWT sem bloquear o pedido caso falhe, sendo utilizado em rotas públicas que apresentam conteúdo diferente para utilizadores autenticados. O `anonymousUsersOnly` impede utilizadores já autenticados de aceder a *endpoints* de registo e *login*.

8.3.2 Filtragem de Dados por Papel

A autorização não se limita a bloquear ou permitir *endpoints* — a filtragem de dados é aplicada ao nível da *query* SQL. Quando um Consultor lista as suas candidaturas, a *query* inclui automaticamente a restrição ao seu identificador; quando um Service Line Leader consulta candidaturas, é adicionado um filtro pela sua Service Line. Esta abordagem garante que dados não autorizados nunca transitam pela rede, mesmo em caso de erro na lógica do controlador.

8.3.3 Guardas de Rota no Frontend

No lado do cliente, o sistema implementa três níveis de proteção. O `ProtectedRoute` verifica se o utilizador está autenticado, redirecionando para a página de *login* caso contrário, e suporta renderização otimista durante o carregamento quando existe uma sessão registada em `localStorage`, evitando um *flash* do ecrã de carregamento. O `RoleRoute` verifica se o papel do utilizador está incluído nos papéis permitidos, renderizando uma página de erro 403 caso contrário. O `PublicRoute` envolve as rotas de autenticação, impedindo o acesso por utilizadores já autenticados.

8.4 Validação e Sanitização de Dados de Entrada

A validação constitui uma barreira dupla — implementada tanto no *backend* como no *frontend* — que rejeita dados não conformes antes de atingirem a lógica de negócio ou a base de dados.

8.4.1 Esquemas Zod no Backend

A validação no servidor está centralizada na diretória `validations/` e utiliza a biblioteca Zod [`zod`] para definição de esquemas com tipagem estrita. Um *middleware* dedicado

intercepta os erros de validação e devolve respostas HTTP 400 com erros detalhados por campo, padronizados através de um mapa de códigos de erro que traduz os códigos internos do Zod para códigos legíveis por máquina.

O ficheiro de regras partilhadas (`shared-rules.js`) define funções de validação reutilizáveis para campos comuns: nomes completos com 2 a 255 caracteres, sanitização HTML e formatação de nome próprio; *usernames* alfanuméricos de 3 a 50 caracteres; *emails* com formato válido; *passwords* de 8 a 100 caracteres com obrigatoriedade de maiúscula, minúscula, número e carácter especial; números de telefone em formato internacional (7 a 15 dígitos); datas de nascimento com validação de idade mínima de 16 anos; e biografias com limites de 5000 caracteres ou 500 palavras.

Os esquemas por domínio agrupam estas regras em validações contextuais. Destaca-se o esquema de registo, que utiliza uma *discriminated union* por papel, onde os campos obrigatórios variam consoante se trate de Consultor, Talent Manager ou Service Line Leader.

8.4.2 Validação de Upload de Ficheiros

A segurança no *upload* de ficheiros aplica quatro controlos sequenciais: verificação dos tipos MIME permitidos (PDF, JPEG, PNG, ZIP), limite de tamanho de 10 MB, obrigatoriedade de extensão válida no nome do ficheiro, e validação prévia ao URL assinado que impede *uploads* de ficheiros excessivos antes mesmo da geração do URL. Os URLs de imagem de perfil são validados contra o caminho de armazenamento temporário do Supabase, impedindo a injeção de URLs externos.

8.4.3 Validação no Frontend

No lado do cliente, a validação está centralizada na pasta `validations/` e assenta em dois *hooks* React complementares. O `useFormValidation` gere o estado do formulário sem bibliotecas externas, mantendo valores, campos tocados, erros e a *flag* de tentativa de submissão, com lógica de visibilidade que apenas apresenta erros quando o campo foi tocado ou a submissão foi tentada. A extensão `useFormWithServerErrors` mescla erros de validação do cliente com erros campo-a-campo retornados pelo servidor, dando prioridade aos erros do cliente e limpando automaticamente os erros do servidor quando o utilizador interage com o campo afetado. Os campos com erros recebem bordo e fundo na cor de erro, com mensagem descritiva em texto vermelho.

8.5 Estratégia de Cache com Redis

A performance da API é sustentada por uma camada de *cache* centralizada que abstrai a complexidade do armazenamento temporário por detrás de uma interface uniforme de três operações fundamentais: `get`, `set` e `invalidate`.

8.5.1 Arquitetura e Fallback

Em ambiente de produção, a camada utiliza a biblioteca `ioredis` para comunicar com uma instância Redis [**redis**] externa, tirando partido da sua velocidade de leitura sub-milissegundo. A estratégia de conexão inclui *backoff* exponencial com atraso máximo de 2000 ms e 3 tentativas por pedido. Em ambientes de desenvolvimento ou cenários onde o Redis não esteja disponível, o módulo degrada graciosamente para um *fallback* em memória baseado em Map, assegurando que a API continua funcional sem dependências externas obrigatórias. Em produção, a presença do Redis é obrigatória para funcionalidades críticas como limitação de taxa e bloqueio de contas.

8.5.2 Padrão Cache-Aside

A utilização nos controladores segue consistentemente a estratégia *cache-aside*: o controlador verifica primeiro se existe uma entrada válida em *cache* para a chave correspondente ao pedido; se existir, devolve imediatamente os dados cacheados sem consultar a base de dados; caso contrário, executa a *query* ao PostgreSQL [**postgresql**], armazena o resultado com um TTL apropriado, e só então devolve a resposta. Este padrão reduz drasticamente a carga sobre a base de dados em cenários de leitura repetida, que constituem a maioria dos pedidos da plataforma.

8.5.3 Construção de Chaves e Invalidação

As chaves de *cache* seguem uma convenção hierárquica baseada em prefixos. A listagem de badges, por exemplo, gera chaves no formato `badge:list:{hash}`, onde o *hash* incorpora todos os filtros e parâmetros de paginação aplicados. O perfil do utilizador autenticado é cacheado sob a chave `user:profile:{user_id}`.

A invalidação segue um modelo por prefixo: quando ocorre uma operação de escrita que altera o estado de um domínio, o controlador invalida todas as chaves cujo prefixo corresponda ao domínio afetado. Esta abordagem elimina de uma só vez todas as variantes de listagem cacheadas, independentemente da combinação específica de filtros que as gerou. O módulo de avisos destaca-se pela implementação de TTL dinâmico, calculado com base na próxima data de alteração de visibilidade dos avisos ativos — quando um aviso entra ou sai do período de vigência, o *cache* é automaticamente invalidado sem intervenção manual.

8.6 Comunicação em Tempo Real

A comunicação em tempo real assenta no Socket.IO [**socketio**], partilhando o porto HTTP com o Express [**expressjs**] através de um servidor HTTP único. Esta decisão arquitetural elimina a necessidade de infraestrutura adicional para suportar WebSockets.

8.6.1 Autenticação e Salas

A autenticação do WebSocket verifica o JWT durante o *handshake* inicial, extraíndo o token do objeto `socket.handshake.auth.token`. Conexões sem token válido são rejeitadas com códigos de erro específicos (`AUTH_TOKEN_NOT_PROVIDED` ou `AUTH_TOKEN_INVALID`). Cada utilizador autenticado é automaticamente associado a uma sala privada `user:{userId}`, permitindo o envio direcionado de eventos através da função `emitToUser(userId, event, data)`, que transmite para todos os dispositivos conectados do utilizador.

8.6.2 Reconexão e Resiliência

No lado do cliente, o Socket.IO está configurado com reconexão automática: tentativas infinitas, atraso inicial de 1 segundo com crescimento exponencial até um máximo de 30 segundos. A cada tentativa de reconexão, o *socket* obtém o token JWT mais recente do interceptor Axios [**axios**], garantindo que reconexões após um *refresh* de token utilizam credenciais atualizadas. Se o Socket.IO não estiver disponível no servidor, o WebSocket é desativado graciosamente sem afetar o funcionamento da API REST.

Os eventos tratados no *frontend* incluem `notification:new` (nova notificação adicionada ao topo da lista com incremento de contadores), `notification:read` (atualização de estado e decremento de contador) e `notification:all-read` (reinício de contadores). A receção de uma notificação pode desencadear a atualização automática de dados noutras páginas — por exemplo, a lista de candidaturas é refrescada quando chega uma notificação de validação.

8.7 Tarefas Agendadas em Segundo Plano

A API inclui quatro *workers* agendados via node-cron [**nodecron**] que executam em paralelo com o servidor HTTP, inicializados automaticamente durante o arranque da aplicação.

O *worker* de expiração de badges executa a cada hora, identificando badges cuja data de expiração se aproxima e enviando alertas progressivos nos limiares de 30, 7 e 1 dia, com notificação final no momento da expiração. A idempotência é garantida pelo registo do último limiar notificado, impedindo duplicação de alertas.

Os *workers* de SLA executam a cada 15 minutos em duas variantes: o *worker* global identifica candidaturas nos estados `Submitted` ou `In validation` cujo tempo de permanência excede o tempo de resposta configurado no SLA aplicável; o *worker* personalizado aplica a mesma lógica para regras específicas por Service Line. Ambos verificam na tabela `sla_breach_alerts` se a combinação (SLA, candidatura, utilizador) já foi notificada, e um índice único composto garante a unicidade a nível da base de dados.

O *worker* de lembretes de objetivos executa a cada hora, identificando metas cuja data-alvo se aproxima (7 dias de antecedência) e enviando notificações e emails aos consultores.

Todos os alertas são entregues simultaneamente através dos três canais disponíveis — notificação *in-app* via WebSocket, notificação *push* via Firebase [**firebase**] e email via Nodemailer [**nodemailer**] — conforme as preferências configuradas.

8.8 Armazenamento de Ficheiros

O armazenamento de ficheiros integra com o Supabase Storage [**supabase**], utilizando dois *buckets* com finalidades distintas: *public-assets* para imagens de perfil e badges com URLs públicos, e *private-assets* para ficheiros de evidência com URLs assinados de curta duração.

O fluxo de *upload* segue um padrão de URL pré-assinado que elimina a necessidade de os ficheiros transitarem pelo servidor da API. O cliente valida localmente o ficheiro (tipo MIME, extensão e tamanho), obtém um URL de *upload* assinado com validade de 5 minutos através da API, envia o ficheiro diretamente para o Supabase via pedido HTTP, e comunica à API o caminho do ficheiro para associação ao registo correspondente. Os nomes de ficheiro são tornados únicos pela adição de um *timestamp*, e cada segmento do caminho é codificado para URL.

Os ficheiros são inicialmente carregados para uma pasta temporária (*temp/*). Quando a operação é confirmada (por exemplo, a submissão de uma candidatura), o servidor move os ficheiros para a localização permanente. O *download* de ficheiros privados utiliza igualmente URLs assinados com validade de 5 minutos, garantindo que o acesso é temporário e auditável.

O *upload* de buffers gerados pelo servidor (como certificados PDF) é suportado diretamente, com especificação do *content type*. Em ambiente de desenvolvimento sem Supabase configurado, os ficheiros são escritos localmente na pasta `logs/dev_storage` e servidos por uma rota dedicada.

8.9 Sistema de Notificações Multicanal

O sistema de notificações opera em três canais complementares, unificados por uma hierarquia de preferências configurável a dois níveis.

8.9.1 Canais de Entrega

As notificações *in-app* são geridas pela tabela `notifications`, que armazena instâncias concretas com *payload* JSON, URL de ação, tipo categorizado (HOME, BADGES, APPLICATIONS, ACHIEVEMENTS, POINTS, OBJECTIVES, EVOLUTION, ANNOUNCEMENTS, SYSTEM) e *flag* de leitura. A entrega em tempo real ao *frontend* é efetuada através do WebSocket, e o componente `NotificationIcon` na barra superior apresenta um indicador com a contagem de notificações não lidas por tipo, permitindo ao utilizador aceder ao painel

com filtros por tipo e estado de leitura. No menu lateral, cada seção exibe indicadores visuais quando existem notificações não lidas do tipo correspondente.

As notificações *push* para a aplicação móvel são enviadas através do Firebase Cloud Messaging. O sistema gere os tokens FCM na tabela `device_tokens`, detetando e desativando automaticamente tokens obsoletos após falhas de envio (código `messaging/registration-token-not-registered`). O envio para tópicos segmentados por papel é suportado através da função `sendTopicUpdate`.

As notificações por email utilizam Nodemailer [nodemailer] com transporte SMTP Gmail. Cada tipo de email dispõe de *templates* HTML nos três idiomas suportados (pt-PT, en-GB, es-ES), selecionados automaticamente com base no idioma preferido do destinatário. Os *templates* incluem a identidade visual da Softinsa, botões de ação e ligações de *fallback* em texto simples.

8.9.2 Hierarquia de Preferências

As preferências globais, definidas pelo administrador na tabela `notification_preferences`, estabelecem os valores predefinidos para cada tipo de notificação nos campos `is_enabled`, `send_email` e `send_push`. Cada utilizador pode sobrepor estes valores através da tabela `user_notification_preferences`, onde campos com valor NULL herdam automaticamente o valor global. Na ausência de qualquer configuração, todos os canais são considerados ativos por omissão, garantindo que nenhum alerta crítico é silenciado inadvertidamente.

8.10 Internacionalização e Tradução Dinâmica

A plataforma implementa um sistema de internacionalização em duas camadas, cobrindo tanto as *strings* estáticas da interface como os conteúdos dinâmicos da base de dados.

8.10.1 Strings Estáticas com i18next

A internacionalização da interface utiliza o i18next [i18next] com deteção automática do idioma do *browser* e *fallback* para Português. A configuração define dois *namespaces*: `common` para *strings* de UI com aproximadamente 2100 linhas por idioma, e `api` para mensagens de erro traduzidas com cerca de 101 linhas por idioma. Os três idiomas suportados — Português, Inglês e Espanhol — são carregados como importações estáticas de ficheiros JSON. As traduções utilizam interpolação de variáveis e formas plurais para pluralização contextual. A mudança de idioma atualiza imediatamente a interface via `i18next.changeLanguage()`, persiste a preferência no servidor e atualiza o estado local no contexto React.

No *backend*, o idioma preferido de cada utilizador é armazenado como chave estrangeira para a tabela `languages`. Os *templates* de email são codificados por idioma no serviço de email, sendo automaticamente selecionado o *template* correspondente ao idioma do destinatário. Os certificados PDF também são gerados no idioma preferido do consultor.

8.10.2 Conteúdos Dinâmicos com Tradução Automática

Os conteúdos dinâmicos provenientes da base de dados — nomes de badges, descrições, requisitos, títulos de Learning Paths, Service Lines, Áreas e Níveis — são armazenados originalmente em Português e traduzidos automaticamente em tempo de execução. A API integra com a Google Cloud Translation API [**googletranslate**] através de um serviço dedicado que implementa várias otimizações: deteção automática do idioma de origem pela biblioteca *franc-min* (com mínimo de 15 caracteres para deteção fiável), *cache* Redis com TTL de 7 dias indexada por *hash* MD5 do conteúdo e idioma, e tradução em lote com envio de apenas os textos não cacheados numa única chamada. Em caso de falha na API externa, o serviço degrada graciosamente devolvendo os textos originais.

No *frontend*, o *TranslationContext* gere uma fila de textos pendentes com *batching* temporal de 60 milissegundos, agregando múltiplos textos individuais num único pedido HTTP. O componente *TranslatedText* encapsula qualquer texto dinâmico, exibindo o texto original enquanto a tradução é carregada e substituindo-o de forma transparente assim que a *Promise* é resolvida. Um *cache* local em memória, indexado por idioma e texto original, evita chamadas repetidas para textos já traduzidos na sessão corrente. Quando o idioma ativo é Português, nenhuma tradução é efetuada.

8.11 Logging e Rastreabilidade de Pedidos

O sistema de *logging* utiliza a biblioteca Winston, configurada com transportes diferenciados por ambiente: em produção, apenas consola com nível configurável via variável de ambiente; em desenvolvimento, ficheiro adicional (*logs/api-debug.log*) em nível *debug*.

O *middleware* de *logging* de pedidos regista cada pedido e resposta com detalhes operacionais: método HTTP, URL, endereço IP, *user-agent*, parâmetros de *query*, corpo do pedido, código de estado, tempo de resposta em milissegundos e tamanho da resposta. Cada pedido recebe um identificador único (*X-Request-ID*) para rastreabilidade.

A proteção de dados sensíveis é implementada por uma função de redação que substitui automaticamente os valores dos campos sensíveis — *password*, *token*, *authorization*, *secret*, *apikey*, *api_key*, *access_token*, *refresh_token*, *cookie* e *set-cookie* — pelo texto [REDACTED]. A função trata também referências circulares, apresenta o tamanho de *buffers* sem expor o conteúdo, e trunca *strings* superiores a 2000 caracteres. Pedidos abortados pelo cliente são igualmente detetados e registados.

8.12 Gestão de Estado no Frontend

A aplicação web utiliza cinco contextos React [**react**] como mecanismo principal de gestão de estado global, aninhados numa ordem deliberada no ficheiro *App.jsx* para garantir

que cada *provider* tem acesso aos dados dos *providers* exteriores.

O `AuthContext` é responsável pelo ciclo de vida da sessão de autenticação, gerindo o token JWT em memória, o estado de autenticação, o mecanismo de *refresh* automático e a monitorização de atividade com os dois modos de sessão descritos na secção de autenticação.

O `UserContext` gere o perfil do utilizador autenticado, os pontos de gamificação (para Consultores) e todo o subsistema de notificações. Integra-se com o `WebSocket` para receber notificações em tempo real, mantendo contadores por tipo e expondo métodos para marcar notificações como lidas, buscar notificações paginadas e atualizar o perfil. A última notificação recebida é exposta a páginas subscritoras para desencadear *re-fetches* reativos.

O `LanguageContext` carrega e armazena em *cache* a lista de idiomas disponíveis da API, com memoização de `Promise` para garantir uma única chamada por ciclo de vida. O `TranslationContext` gere o sistema de tradução dinâmica de conteúdos da base de dados, conforme descrito na secção de internacionalização. O `GdprConsentContext` centraliza o fluxo de consentimento de privacidade, expondo um método `requestConsent()` que apresenta o modal de aceitação da política quando necessário e confirma automaticamente quando o consentimento já foi previamente concedido.

A comunicação HTTP com a API está centralizada numa instância `Axios` [**axios**] com interceptor de pedido que injeta automaticamente o cabeçalho de autorização. O interceptor de resposta implementa *retry* automático para respostas 401: marca o pedido com *flag* de *retry* para evitar ciclos infinitos, tenta um *refresh* do token, reenvia o pedido original com o novo token em caso de sucesso, ou dispara um evento global de *logout* (`auth:logout`) em caso de falha.

8.13 Conformidade com o RGPD

A conformidade com o Regulamento Geral sobre a Proteção de Dados [**rgpd**] é assegurada por um módulo dedicado que abrange políticas, consentimentos, exportação de dados e eliminação de conta.

As políticas são geridas na tabela `gdpr_policies` com suporte a três tipos (Privacidade, Termos e *Cookies*), cada um com versionamento semântico e *flag* de obrigatoriedade. A comparação de versões permite que apenas a versão mais recente de cada tipo permaneça ativa, com reconciliação automática. As políticas obrigatórias são automaticamente aceites durante o registo do utilizador, com registo na tabela `gdpr_consent_history`.

O histórico de consentimentos é um registo imutável que documenta cada ação (ACCEPTED ou REVOKED) com *timestamp*, endereço IP, *user-agent* e versão da política. O campo `gdpr_accepted` na tabela de consultores permite rastreio rápido do estado de consentimento. A aplicação web implementa um *gate* de consentimento que verifica a versão da política aceite antes de permitir ações como publicação de badges ou partilha de perfil público.

O módulo disponibiliza ainda a exportação de todos os dados pessoais em formato

JSON e um *endpoint* de eliminação de conta (/api/gdpr/delete-account), permitindo ao utilizador exercer o seu direito ao esquecimento com *logout* automático subsequente.

8.14 Verificação Pública de Credenciais e Certificados

Cada badge atribuído recebe automaticamente uma ligação pública de verificação (`public_verification`) gerada no momento da aceitação da candidatura. Esta ligação constitui um URL único que permite a qualquer pessoa — sem necessidade de autenticação — confirmar a autenticidade de uma credencial, visualizando o título do badge, a descrição, os pontos, o nome do detentor e as datas de atribuição e expiração. A proteção contra XSS é assegurada pelo escapamento de todos os valores controlados pelo utilizador antes da renderização HTML.

A geração de certificados em formato PDF utiliza a biblioteca PDFKit [**pdfkit**], compondo documentos personalizados em orientação paisagem A4 com dupla moldura, título do badge, nome do consultor, descrição, datas de conclusão, nomes dos revisores (Talent Manager e Service Line Leader), código QR de verificação (300×300 píxeis) e rodapé institucional. O conteúdo textual é adaptado ao idioma preferido do consultor. A geração é idempotente — se o certificado já existir para uma candidatura, o sistema devolve o ficheiro existente sem criar duplicado. Os certificados são armazenados no Supabase e disponibilizados através de URLs assinados com validade temporal limitada.

8.15 Arquitetura CSS e Design Responsivo

A estilização da aplicação web segue uma abordagem em três camadas: Bootstrap 5.3 [**bootstrap**] como *framework* de base com *overrides* customizados; um sistema completo de *design tokens* em variáveis CSS globais definidas no ficheiro `colors.css`; e 149 ficheiros CSS Modules com escopo isolado por componente, garantindo zero conflitos de nomes.

O sistema de variáveis CSS define mais de cem propriedades semânticas que abrangem todas as camadas visuais: cores primária (ciano), secundária (azul), erro (vermelho), sucesso (verde) e aviso (âmbar), com variantes de opacidade para estados interativos; gradientes para campos e painéis; cores suaves para cartões; intensidades de *heatmap*; e sombras de cartão. A nomenclatura segue princípios do Material Design 3, com pares de cor semânticos que garantem conformidade WCAG AA de contraste.

A plataforma suporta um sistema de temas dual (claro e escuro), controlado pelo atributo `data-theme` no elemento raiz do documento HTML. Quando o valor `dark` é aplicado, todas as variáveis CSS são redefinidas: os fundos claros são substituídos por tons escuros, as cores de ênfase mantêm a saturação com opacidades ajustadas, e os componentes de visualização de dados dispõem de escalas cromáticas adaptadas. A preferência é persistida em `localStorage` e aplicada instantaneamente sem recarregamento.

A responsividade é garantida desde 300 px de largura, utilizando uma abordagem *mobile-first* com *breakpoints* progressivos: 360 px e 400 px para telefones muito pequenos, 480 px e 576 px para *smartphones* com grelhas de coluna única e botões empilhados, 768 px e 991 px para *tablets* com transição do menu lateral, e 1200 px para *desktop* com grelhas expandidas. O menu lateral comporta-se de forma adaptativa: em *desktop*, apresenta-se como uma barra de ícones colapsável (74 px) expansível para barra completa (280 px); em dispositivos móveis, transforma-se num *drawer* deslizante com *backdrop*, fechando automaticamente após clique numa ligação. A deteção de *breakpoint* é efetuada em tempo real via `window.matchMedia`.

Todas as animações CSS — *spinners*, entrada de *toasts*, *modais*, *skeletons* com efeito *shimmer*, brilho de *badges* especiais e *confetti* de celebração — respeitam a *media query* `prefers-reduced-motion: reduce`, desativando efeitos para utilizadores com sensibilidade a movimento.

9

Considerações Adicionais

Este capítulo documenta funcionalidades transversais e decisões técnicas que complementam as descrições dos capítulos anteriores, abrangendo aspetos de personalização visual, otimização de desempenho, integração com serviços externos e funcionalidades de valor acrescentado que enriquecem a plataforma para além dos requisitos base.

9.1 Modo Escuro e Sistema de Temas

A plataforma implementa um sistema de temas dual (claro e escuro) baseado inteiramente em variáveis CSS (*custom properties*), centralizadas no ficheiro `colors.css`. O ficheiro define mais de cem propriedades semânticas que abrangem todas as camadas visuais da interface: cores de superfície, texto, bordas, *badges*, gráficos, sobreposições e gradientes.

A alternância entre temas é controlada pelo atributo `data-theme` no elemento raiz do documento HTML. Quando o valor `"dark"` é aplicado, todas as variáveis CSS são redefinidas sob o seletor `[data-theme="dark"]`: os fundos claros (`#F7F9FB`) são substituídos por tons escuros (`#0f141a`), as cores de ênfase mantêm a saturação com opacidades ajustadas para garantir contraste adequado, e os componentes de visualização de dados (como os mapas de calor de atividade) dispõem de escalas cromáticas adaptadas. A preferência do utilizador é persistida localmente e aplicada de forma instantânea sem recarregamento da página. Esta dualidade foi considerada desde a fase de prototipagem em Figma [**figma**], onde ambas as variantes foram desenhadas e validadas.

9.2 Estratégia de Cache e Desempenho

O desempenho da API é otimizado através de uma camada de *cache* Redis [**redis**], implementada com a biblioteca `ioredis`. O sistema adota uma estratégia de *fallback* para o ambiente de desenvolvimento, recorrendo a um *mock* em memória quando o Redis não está

disponível; em produção, a sua presença é obrigatória para funcionalidades críticas como limitação de taxa (*rate limiting*) e bloqueio de contas.

A estratégia de *cache* é aplicada seletivamente aos *endpoints* com maior volume de leitura — listagens de avisos, perfis de utilizador, catálogos de badges e definições de SLAs — com TTLs diferenciados por domínio funcional. O módulo de avisos destaca-se pela implementação de TTL dinâmico, calculado com base na próxima data de alteração de visibilidade dos avisos ativos, conforme descrito na Secção 3.4.2. Todas as operações de escrita invalidam o *cache* afetado por prefixo, assegurando a consistência entre os dados em *cache* e a base de dados.

O sistema de tradução dinâmica (descrito na secção seguinte) constitui outro domínio onde o *cache* desempenha um papel central, com entradas de tradução armazenadas com TTL de 7 dias para minimizar chamadas à API externa.

9.3 Tradução Dinâmica de Conteúdos da Base de Dados

Para além das *strings* estáticas da interface (geridas por ficheiros de localização i18next [*i18next*]), todos os conteúdos dinâmicos provenientes da base de dados — nomes de badges, descrições, requisitos, títulos de Learning Paths, Service Lines, Áreas e Níveis — são traduzidos automaticamente em tempo de execução para o idioma selecionado pelo utilizador.

A API integra com a Google Cloud Translation API [*googletranslate*] através de um serviço dedicado (`translation.service.js`) que implementa as seguintes otimizações:

- **Deteção automática de idioma:** A biblioteca *franc-min* identifica o idioma de origem do texto, evitando traduções desnecessárias quando o conteúdo já se encontra no idioma pretendido;
- **Cache com TTL de 7 dias:** Cada tradução é armazenada em Redis [*redis*], eliminando chamadas repetidas à API para o mesmo par (texto, idioma);
- **Tradução em lote (*batching*):** No lado do cliente, a plataforma Web agrega múltiplos textos num único pedido HTTP, reduzindo significativamente o número de chamadas à API e melhorando a latência percebida;
- **Fallback gracioso:** Quando a API de tradução não está disponível, o sistema devolve o texto original, garantindo que a ausência do serviço externo não compromete a utilização da plataforma.

Na aplicação móvel, a tradução *offline* é assegurada pelo ML Kit Translation [*mlkit*], que descarrega modelos de tradução para o dispositivo e processa os textos localmente, alinhado com a arquitetura *offline-first* adotada.

9.4 Exportação de Dados

A plataforma disponibiliza funcionalidades de exportação de dados em três formatos — CSV, XLSX (Excel) e PDF — acessíveis aos perfis de gestão (TM, SLL e Administrador). O módulo de exportação (`exports.controller.js`) implementa a geração de ficheiros a partir dos dados do sistema, suportando os seguintes conjuntos de dados: consultores, badges atribuídos, candidaturas e histórico de pontos.

A exportação respeita o âmbito de acesso de cada perfil: os Service Line Leaders exportam exclusivamente dados da sua Service Line, enquanto os Talent Managers e os Administradores acedem à exportação global. Todos os formatos suportam filtragem por intervalo de datas. Os ficheiros Excel são gerados com a biblioteca ExcelJS, com dimensionamento automático de colunas; os ficheiros PDF são gerados em modo paisagem com PDFKit [**pdfkit**] para acomodar tabelas com muitas colunas; os ficheiros CSV incluem escapamento adequado de campos com caracteres especiais.

9.5 Verificação Pública de Badges e Códigos QR

Cada badge atribuído possui uma ligação pública de verificação (`public_verification_link`), gerada automaticamente no momento da aceitação da candidatura. Esta ligação constitui um URL único que permite a qualquer pessoa — sem necessidade de autenticação — confirmar a autenticidade e os detalhes de uma credencial digital.

A página pública de verificação é servida pelo controlador `public.controller.js` em dois formatos: uma página HTML com renderização completa dos metadados do badge (título, descrição, pontos, nome do detentor, datas de atribuição e expiração) e uma resposta JSON para integração programática com sistemas externos. Complementarmente, o sistema gera códigos QR (300×300 píxeis) para cada badge atribuído, permitindo a incorporação em assinaturas de e-mail corporativo e materiais impressos. A geração de páginas públicas inclui proteção contra XSS através do escapamento de todos os valores controlados pelo utilizador antes da renderização HTML.

9.6 Geração Automática de Certificados

Quando uma candidatura atinge o estado `Accepted`, o sistema disponibiliza a geração automática de um certificado em formato PDF, personalizado com os dados do badge, do consultor e dos validadores envolvidos no processo. O serviço de certificados (`certificate.service.js`) utiliza a biblioteca PDFKit [**pdfkit**] para a composição do documento, que inclui o título do badge, a descrição, a data de atribuição, os nomes dos revisores e o URL de verificação pública.

A geração é idempotente: se o certificado já tiver sido gerado previamente para uma determinada candidatura, o sistema devolve o ficheiro existente sem criar um duplicado. Os certificados são armazenados no serviço de armazenamento Supabase [**supabase**] e disponibilizados para *download* através de URLs assinados com validade temporal limitada. O conteúdo textual do certificado é adaptado ao idioma preferido do consultor (Português, Inglês ou Espanhol).

9.7 Loja de Recompensas por Pontos

O sistema de gamificação é complementado por uma loja de recompensas que permite aos consultores converter os pontos acumulados em benefícios concretos. O administrador pode criar e gerir recompensas (*rewards*) com custo em pontos, descrição e informação de acesso (como ligações para cursos ou códigos de acesso a recursos). As recompensas podem ser ativadas ou desativadas sem remoção dos dados.

Os consultores visualizam as recompensas disponíveis e o seu saldo de pontos atual, podendo efetuar resgates quando dispõem de pontos suficientes. Cada resgate é registado na tabela `reward_redemptions` com rastreabilidade completa, e o consultor recebe uma notificação por e-mail com os detalhes de acesso à recompensa obtida. Este módulo acrescenta uma dimensão tangível ao sistema de pontos, reforçando a motivação para a formação contínua.

10

Conclusão

10.1 Síntese do Trabalho

10.2 Trabalho Futuro
